In [ ]:
```
import numpy as np
```

In [ ]:
```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages
(2.0.0)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages
(from kaggle) (6.3.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.15 in /usr/local/lib/python3.1
2/dist-packages (from kaggle) (0.1.16)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packag
es (from kaggle) (26.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-package
s (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-
packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-p
ackages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-package
s (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (f
rom kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-
packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-pac
kages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-package
s (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/d
ist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
3.12/dist-packages (from requests->kaggle) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pac
kages (from requests->kaggle) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/di
st-packages (from requests->kaggle) (2026.2.25)
```

In [ ]:
```
from google.colab import files
files.upload()
```

In [ ]:
```
!ls
```

```
data.csv  Global_Superstore2.csv  sample_data
```

In [ ]:
```
!unzip global-super-store-dataset.zip
```

```
unzip:  cannot find or open global-super-store-dataset.zip, global-super-store-da
taset.zip.zip or global-super-store-dataset.zip.ZIP.
```

In [ ]:
```
!ls
```

```
data.csv  Global_Superstore2.csv  sample_data
```

In [ ]:
```
import zipfile

# Unzip the downloaded file
with zipfile.ZipFile('global-super-store-dataset.zip', 'r') as zip_ref:
    zip_ref.extractall('dataset')
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2419/888699667.py in <cell line: 0>()
      2
      3 # Unzip the downloaded file
----> 4 with zipfile.ZipFile('global-super-store-dataset.zip', 'r') as zip_ref:
      5     zip_ref.extractall('dataset')

/usr/lib/python3.12/zipfile/__init__.py in __init__(self, file, mode, compressio
n, allowZip64, compresslevel, strict_timestamps, metadata_encoding)
   1350             while True:
   1351                 try:
-> 1352                     self.fp = io.open(file, filemode)
   1353                 except OSError:
   1354                     if filemode in modeDict:

FileNotFoundError: [Errno 2] No such file or directory: 'global-super-store-datas
et.zip'
```

In [ ]:
```python
import os

# List the files in the dataset directory
os.listdir('dataset')
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2419/209768374.py in <cell line: 0>()
      2
      3 # List the files in the dataset directory
----> 4 os.listdir('dataset')

FileNotFoundError: [Errno 2] No such file or directory: 'dataset'
```

In [ ]:
```python
import pandas as pd

try:
    df = pd.read_csv('/content/Global_Superstore2.csv', encoding='latin1')
except UnicodeDecodeError:
    df = pd.read_csv('/content/Global_Superstore2.csv', encoding='ISO-8859-1')
```

In [ ]:
```python
df.head()
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Brisbane
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	Berlin
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	Dakar

5 rows × 24 columns

In [ ]:

```
data = df.copy()
data
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollon
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Bris
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	
...	...	...	...	...	...	...	...	...	
51285	29002	IN-2014-62366	19-06-2014	19-06-2014	Same Day	KE-16420	Katrina Edelman	Corporate	
51286	35398	US-2014-102288	20-06-2014	24-06-2014	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	Ho
51287	40470	US-2013-155768	02-12-2013	02-12-2013	Same Day	LB-16795	Laurel Beltran	Home Office	O
51288	9596	MX-2012-140767	18-02-2012	22-02-2012	Standard Class	RB-19795	Ross Baird	Home Office	Val
51289	6147	MX-2012-134460	22-05-2012	26-05-2012	Second Class	MC-18100	Mick Crebagga	Consumer	Tip

51290 rows × 24 columns

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 24 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Row ID          51290 non-null  int64
 1   Order ID        51290 non-null  object
 2   Order Date      51290 non-null  object
 3   Ship Date       51290 non-null  object
 4   Ship Mode       51290 non-null  object
 5   Customer ID     51290 non-null  object
 6   Customer Name   51290 non-null  object
 7   Segment         51290 non-null  object
 8   City            51290 non-null  object
 9   State           51290 non-null  object
 10  Country         51290 non-null  object
 11  Postal Code     9994 non-null   float64
 12  Market          51290 non-null  object
 13  Region          51290 non-null  object
 14  Product ID      51290 non-null  object
 15  Category        51290 non-null  object
 16  Sub-Category    51290 non-null  object
 17  Product Name    51290 non-null  object
 18  Sales           51290 non-null  float64
 19  Quantity        51290 non-null  int64
 20  Discount        51290 non-null  float64
 21  Profit          51290 non-null  float64
 22  Shipping Cost   51290 non-null  float64
 23  Order Priority  51290 non-null  object
dtypes: float64(5), int64(2), object(17)
memory usage: 9.4+ MB
```

In [ ]:
```
data['Market'].value_counts()
```

Out[ ]:

Market	count
APAC	11002
LATAM	10294
EU	10000
US	9994
EMEA	5029
Africa	4587
Canada	384

dtype: int64

In [ ]:
```
data['City'].value_counts()
```

Out[ ]:

City	count
New York City	915
Los Angeles	747
Philadelphia	537
San Francisco	510
Santo Domingo	443
...	...
Stadtlohn	1
Elyria	1
Layton	1
Tartagal	1
Vanderbijlpark	1

3636 rows × 1 columns

dtype: int64

In [ ]:
```
data['Country'].value_counts()
```

Out[ ]:

Country	count
United States	9994
Australia	2837
France	2827
Mexico	2644
Germany	2065
...	...
South Sudan	2
Bahrain	2
Swaziland	2
Burundi	2
Eritrea	2

147 rows × 1 columns

dtype: int64

In [ ]:
```
data['Region'].value_counts()
```

Out[ ]:

Region	count
Central	11117
South	6645
EMEA	5029
North	4785
Africa	4587
Oceania	3487
West	3203
Southeast Asia	3129
East	2848
North Asia	2338
Central Asia	2048
Caribbean	1690
Canada	384

dtype: int64

In [ ]:
```
data.isnull().sum()
```

Out[ ]:

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
City	0
State	0
Country	0
Postal Code	41296
Market	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
Shipping Cost	0
Order Priority	0

dtype: int64

In [ ]:
```
data['Postal Code'] =data['Postal Code'].fillna('unknown')
```

In [ ]:
```
data.isnull().sum()
```

Out[ ]:

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
City	0
State	0
Country	0
Postal Code	41296
Market	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
Shipping Cost	0
Order Priority	0

dtype: int64

In [ ]:
```
data.describe()
```

Out[ ]:

	Row ID	Sales	Quantity	Discount	Profit	Shippi Co
count	51290.00000	51290.000000	51290.000000	51290.000000	51290.000000	51290.0000
mean	25645.50000	246.490581	3.476545	0.142908	28.610982	26.3759
std	14806.29199	487.565361	2.278766	0.212280	174.340972	57.2968
min	1.00000	0.444000	1.000000	0.000000	-6599.978000	0.0000
25%	12823.25000	30.758625	2.000000	0.000000	0.000000	2.6100
50%	25645.50000	85.053000	3.000000	0.000000	9.240000	7.7900
75%	38467.75000	251.053200	5.000000	0.200000	36.810000	24.4500
max	51290.00000	22638.480000	14.000000	0.850000	8399.976000	933.5700

In [ ]:

```
data.nunique()
```

Out[ ]:

	0
Row ID	51290
Order ID	25035
Order Date	1430
Ship Date	1464
Ship Mode	4
Customer ID	1590
Customer Name	795
Segment	3
City	3636
State	1094
Country	147
Postal Code	632
Market	7
Region	13
Product ID	10292
Category	3
Sub-Category	17
Product Name	3788
Sales	22995
Quantity	14
Discount	27
Profit	24575
Shipping Cost	10037
Order Priority	4

dtype: int64

In [ ]:

```
data['Ship Mode'].value_counts()
```

Out[ ]:

Ship Mode	count
Standard Class	30775
Second Class	10309
First Class	7505
Same Day	2701

dtype: int64

In [ ]:
```
data['Category'].value_counts()
```

Out[ ]:

Category	count
Office Supplies	31273
Technology	10141
Furniture	9876

dtype: int64

In [ ]:
```
data['Order Date'] = pd.to_datetime(data['Order Date'])
data['Ship Date'] = pd.to_datetime(data['Ship Date'])
```

```
/tmp/ipykernel_2419/2887490760.py:1: UserWarning: Parsing dates in %d-%m-%Y forma
t when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specif
y a format to silence this warning.
  data['Order Date'] = pd.to_datetime(data['Order Date'])
/tmp/ipykernel_2419/2887490760.py:2: UserWarning: Parsing dates in %d-%m-%Y forma
t when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specif
y a format to silence this warning.
  data['Ship Date'] = pd.to_datetime(data['Ship Date'])
```

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 24 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Row ID          51290 non-null  int64
 1   Order ID        51290 non-null  object
 2   Order Date      51290 non-null  datetime64[ns]
 3   Ship Date       51290 non-null  datetime64[ns]
 4   Ship Mode       51290 non-null  object
 5   Customer ID     51290 non-null  object
 6   Customer Name   51290 non-null  object
 7   Segment         51290 non-null  object
 8   City            51290 non-null  object
 9   State           51290 non-null  object
 10  Country         51290 non-null  object
 11  Postal Code     51290 non-null  object
 12  Market          51290 non-null  object
 13  Region          51290 non-null  object
 14  Product ID      51290 non-null  object
 15  Category        51290 non-null  object
 16  Sub-Category    51290 non-null  object
 17  Product Name    51290 non-null  object
 18  Sales           51290 non-null  float64
 19  Quantity        51290 non-null  int64
 20  Discount        51290 non-null  float64
 21  Profit          51290 non-null  float64
 22  Shipping Cost   51290 non-null  float64
 23  Order Priority  51290 non-null  object
dtypes: datetime64[ns](2), float64(4), int64(2), object(16)
memory usage: 9.4+ MB
```

Basic analysis based on Sales

In [ ]:
```
top_countries = data.groupby('Country')['Sales'].sum().sort_values(ascending=Fal
top_countries
```

Out[ ]:

Country	Sales
United States	2.297201e+06
Australia	9.252359e+05
France	8.589311e+05
China	7.005620e+05
Germany	6.288400e+05

dtype: float64

In [ ]:
```
# what are the top 5 cities with the highest total profit
top_cites = data.groupby('City')['Profit'].sum().sort_values(ascending=False).he
top_cites
```

Out[ ]:

City	Profit
New York City	62036.98370
Los Angeles	30440.75790
Seattle	29156.09670
Managua	17853.71804
San Francisco	17507.38540

dtype: float64

In [ ]:
```
#What are the top 5 product categories with the highest total sales?
top_categories = data.groupby('Category')['Sales'].sum().sort_values(ascending=F
top_categories
```

Out[ ]:

Category	Sales
Technology	4.744557e+06
Furniture	4.110874e+06
Office Supplies	3.787070e+06

dtype: float64

In [ ]:
```
#Find the top 5 most profitable products and their total profit
top_products = data.groupby('Product Name')['Profit'].sum().sort_values(ascendin
top_products
```

Out[ ]:

Product Name	Profit
Canon imageCLASS 2200 Advanced Copier	25199.9280
Cisco Smart Phone, Full Size	17238.5206
Motorola Smart Phone, Full Size	17027.1130
Hoover Stove, Red	11807.9690
Sauder Classic Bookcase, Traditional	10672.0730

dtype: float64

In [ ]:
```
data.head(1)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City	State	..
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City	New York	.

1 rows × 24 columns

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 24 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Row ID          51290 non-null  int64
 1   Order ID        51290 non-null  object
 2   Order Date      51290 non-null  datetime64[ns]
 3   Ship Date       51290 non-null  datetime64[ns]
 4   Ship Mode       51290 non-null  object
 5   Customer ID     51290 non-null  object
 6   Customer Name   51290 non-null  object
 7   Segment         51290 non-null  object
 8   City            51290 non-null  object
 9   State           51290 non-null  object
 10  Country         51290 non-null  object
 11  Postal Code     51290 non-null  object
 12  Market          51290 non-null  object
 13  Region          51290 non-null  object
 14  Product ID      51290 non-null  object
 15  Category        51290 non-null  object
 16  Sub-Category    51290 non-null  object
 17  Product Name    51290 non-null  object
 18  Sales           51290 non-null  float64
 19  Quantity        51290 non-null  int64
 20  Discount        51290 non-null  float64
 21  Profit          51290 non-null  float64
 22  Shipping Cost   51290 non-null  float64
 23  Order Priority  51290 non-null  object
dtypes: datetime64[ns](2), float64(4), int64(2), object(16)
memory usage: 9.4+ MB
```

In [ ]:
```
data['month'] = data['Order Date'].dt.month
```

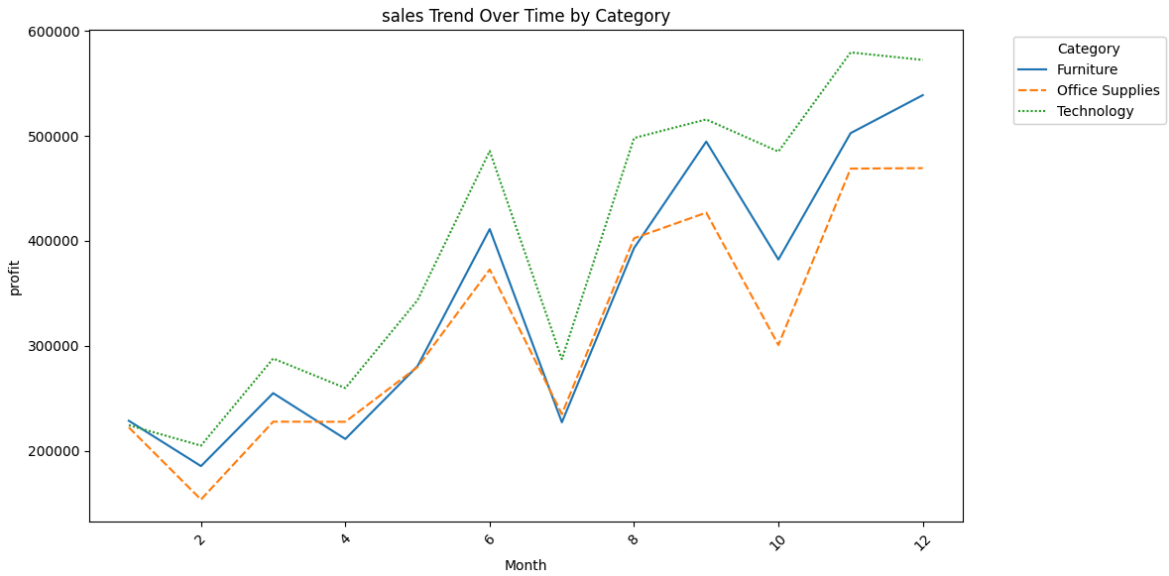
In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:

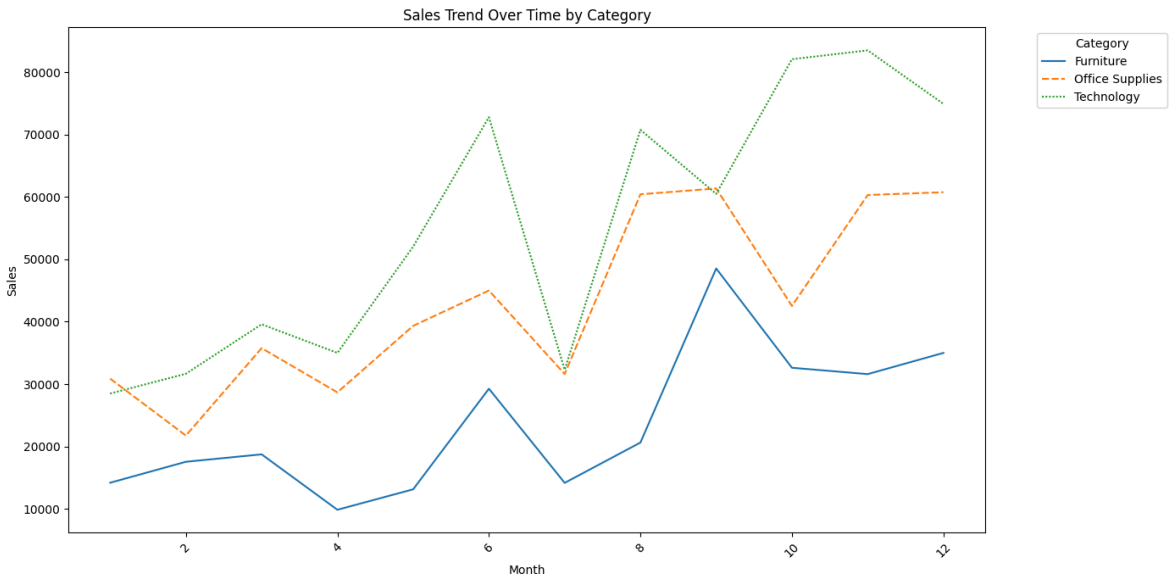
In [ ]:
```
#How do sales and profit trends differ by product category    over time?
sales_trends = data.groupby(['month', 'Category'])['Sales'].sum().reset_index()
profit_trends = data.groupby(['month', 'Category'])['Profit'].sum().reset_index(
```

In [ ]:
```
# Pivot tables for better plotting
sales_pivot = sales_trends.pivot(index='month', columns='Category', values='Sale
profit_pivot = profit_trends.pivot(index='month', columns='Category', values='Pr
```

In [ ]:
```
# Plot Sales Trend
plt.figure(figsize=(12, 6))
sns.lineplot(data=sales_pivot)
plt.title('sales Trend Over Time by Category')
plt.xlabel('Month')
plt.ylabel('profit')
plt.legend(title='Category',bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
plt.figure(figsize=(14, 7))
sns.lineplot(data=profit_pivot)
plt.title('Sales Trend Over Time by Category')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend(title='Category', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
data.head(2)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	2013-02-05	2013-02-07	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong

2 rows × 25 columns

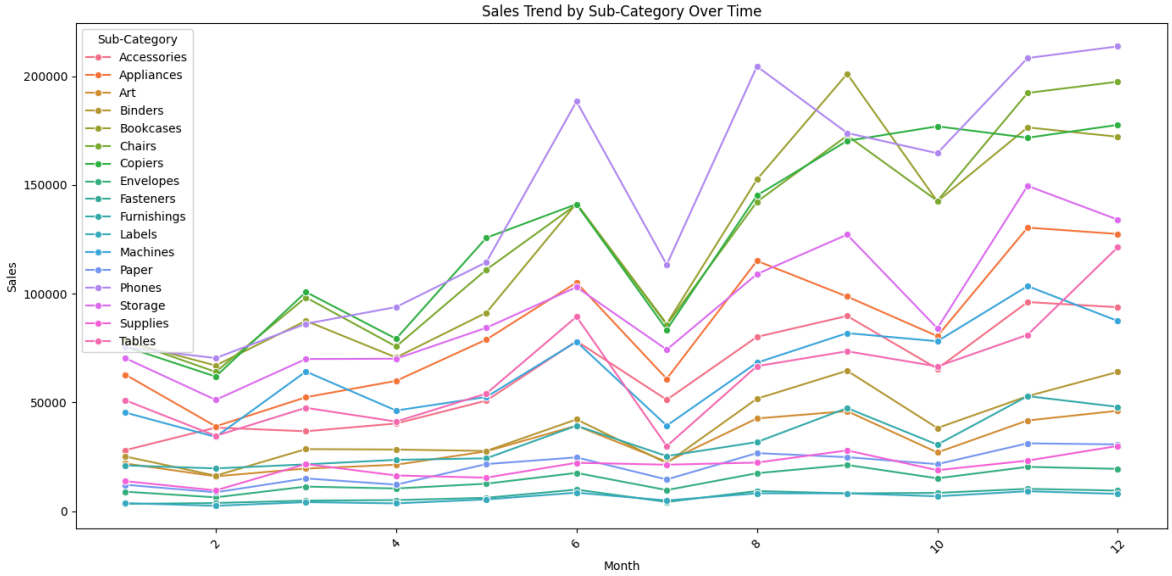
In [ ]:
```
#How do sales and profit trends differ by product sub-category over time?
sub_sales_trend = data.groupby(['month' , 'Sub-Category'])['Sales'].sum().reset_
sub_profit_trend = data.groupby(['month' , 'Sub-Category'])['Profit'].sum().rese
```

In [ ]:
```
#pivoting
sub_sales_pivot = sub_sales_trend.pivot(index='month' , columns='Sub-Category' ,
sub_profit_pivot = sub_profit_trend.pivot(index='month' , columns='Sub-Category'
```

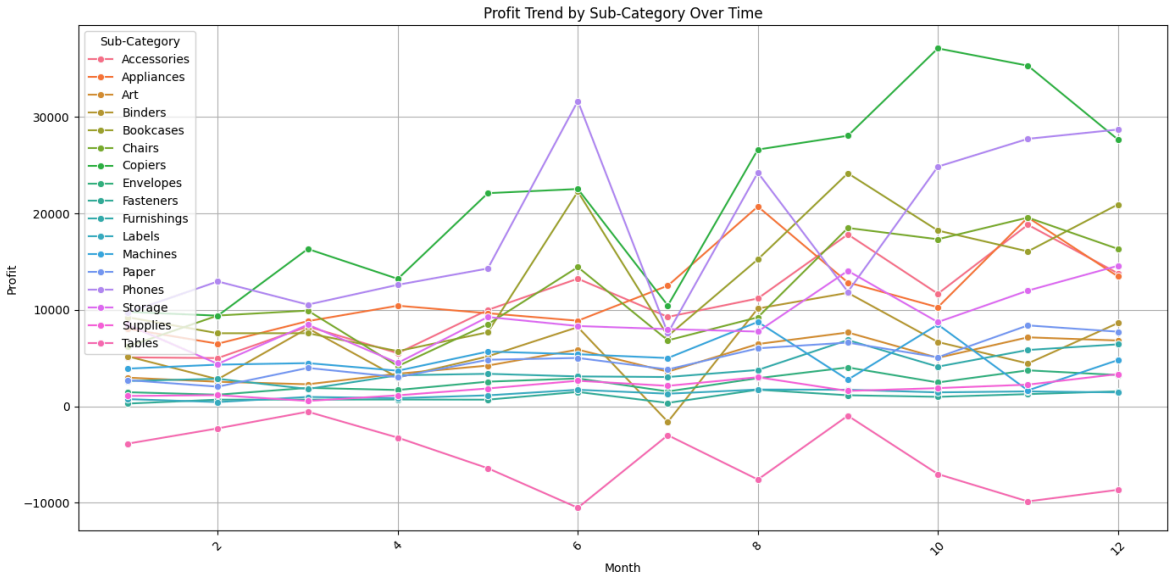
In [ ]:
```
#plotting1
sub_sales_trend.reset_index(inplace=True)
sub_profit_trend.reset_index(inplace=True)
```

In [ ]:
```
# Plot Sales Trend by Sub-Category
plt.figure(figsize=(14, 7))
sns.lineplot(data=sub_sales_trend, x='month' , y='Sales' ,hue='Sub-Category',mar
plt.title('Sales Trend by Sub-Category Over Time')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend(title='Sub-Category')
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:
```
# Plot Profit Trend by Sub-Category
plt.figure(figsize=(14, 7))
sns.lineplot(data=sub_profit_trend, x='month', y='Profit', hue='Sub-Category', m
plt.title('Profit Trend by Sub-Category Over Time')
plt.xlabel('Month')
plt.ylabel('Profit')
plt.legend(title='Sub-Category')
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```



In [ ]:
```
data.head(1)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City	State	..
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City	New York	.

1 rows × 25 columns

In [ ]:
```
#Understanding co relation between discount levels and Profit Margins
data['Profit Margin'] = data['Profit'] / (data['Sales'])
# Calculate correlation between discount and profit margin
correlation = data[['Discount', 'Profit Margin']].corr().iloc[0, 1]
print(f"Correlation between Discount and Profit Margin: {correlation:.2f}")
```

```
Correlation between Discount and Profit Margin: -0.85
```

In [ ]:
```
data['Country'].nunique()
```

Out[ ]: 147

In [ ]:
```
data.head(1)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City	State	..
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City	New York	.

1 rows × 26 columns

In [ ]:
```
City_Sales = data.groupby('City')['Sales'].agg(['sum', 'mean']).reset_index()
City_Sales.columns = ['City', 'Total Sales', 'Average Sales']
```

In [ ]:
```
# Total and average sales for each state
State_sales = data.groupby('State')['Sales'].agg(['sum', 'mean']).reset_index()
State_sales.columns = ['State', 'Total Sales', 'Average Sales']
```

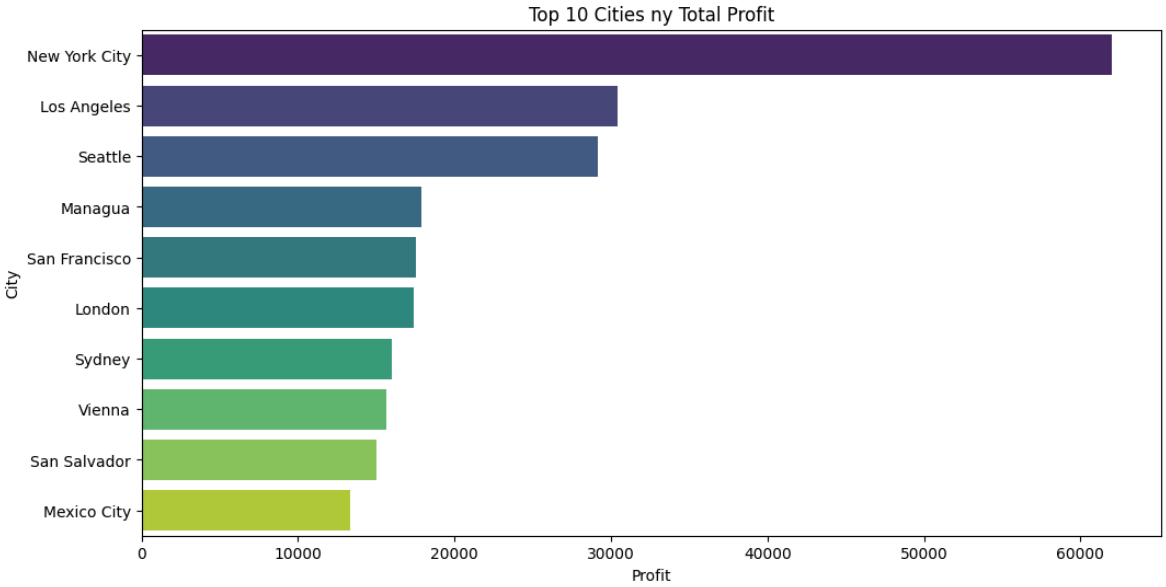
In [ ]:
```
# Total and average sales for each country
Country_sales = data.groupby('Country')['Sales'].agg(['sum', 'mean']).reset_inde
Country_sales.columns = ['Country', 'Total Sales', 'Average Sales']
```

In [ ]:
```
# Total and average sales for each region
Region_sales = data.groupby('Region')['Sales'].agg(['sum', 'mean']).reset_index(
Region_sales.columns = ['Region', 'Total Sales', 'Average Sales']
```

In [ ]:
```
#Objective: Understand the total profit and profit margins across different loca
  #city
  # Total profit for each city
city_profit = data.groupby('City')['Profit'].sum().reset_index().sort_values(by=
```

In [ ]:
```
# Visualize the top and bottom cities by profit
plt.figure(figsize=(12, 6))
sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='viridis')
plt.title('Top 10 Cities ny Total Profit')
plt.show()
```

```
/tmp/ipykernel_556/4020265858.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='viridis')
```



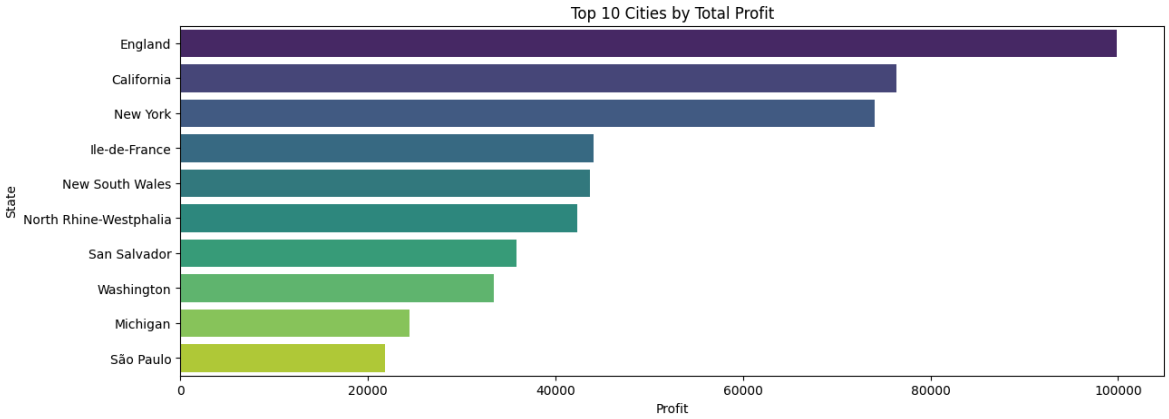
In [ ]:
```
plt.figure(figsize=(14, 7))
sns.barplot(x='Profit', y='City', data=city_profit.tail(10), palette='magma')
plt.title('Bottom 10 Cities by Total Profit')
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2419/1623679193.py in <cell line: 0>()
----> 1 plt.figure(figsize=(14, 7))
      2 sns.barplot(x='Profit', y='City', data=city_profit.tail(10), palette='mag
ma')
      3 plt.title('Bottom 10 Cities by Total Profit')
      4 plt.show()

NameError: name 'plt' is not defined
```

In [ ]:
```
state_profit = data.groupby('State')['Profit'].sum().reset_index().sort_values(b
```

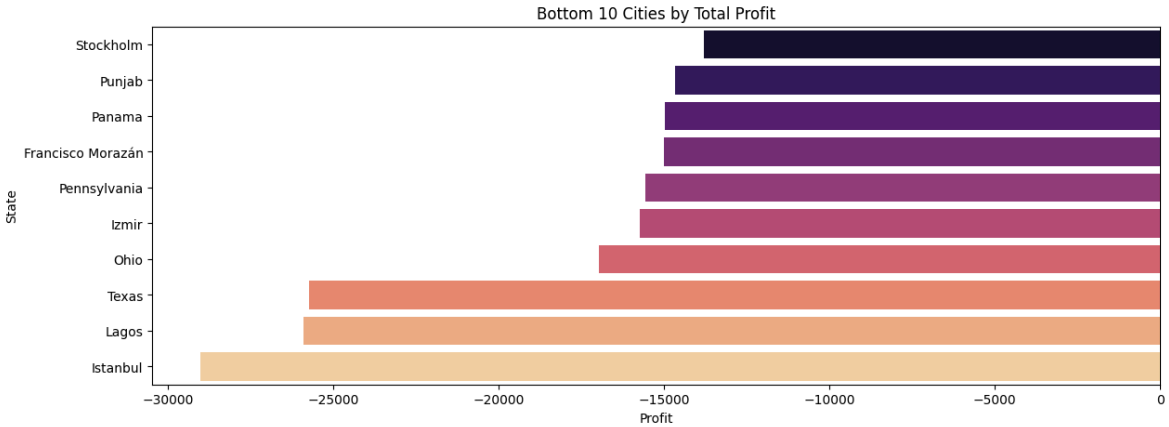
In [ ]:
```
plt.figure(figsize=(14, 5))
sns.barplot(x='Profit', y='State', data=state_profit.head(10), palette='viridis'
plt.title('Top 10 Cities by Total Profit')
plt.show()
```

```
/tmp/ipykernel_556/2921980172.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='State', data=state_profit.head(10), palette='viridi
s')
```



In [ ]:
```
plt.figure(figsize=(14, 5))
sns.barplot(x='Profit', y='State', data=state_profit.tail(10), palette='magma')
plt.title('Bottom 10 Cities by Total Profit')
plt.show()
```

```
/tmp/ipykernel_556/4007313372.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='State', data=state_profit.tail(10), palette='magma')
```

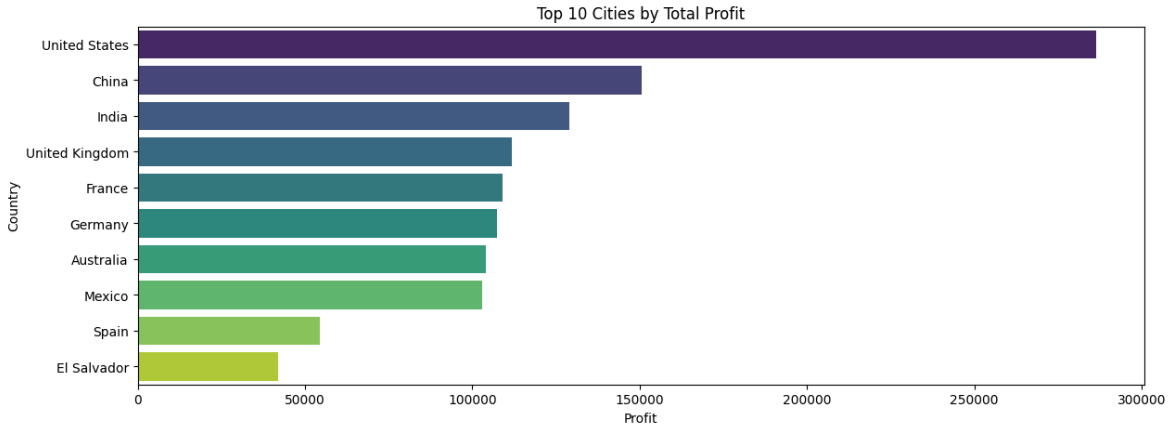


In [ ]:
```
#country
# Total profit for each Country
Country_profit = data.groupby('Country')['Profit'].sum().reset_index().sort_valu
```

In [ ]:
```
# Visualize the top and bottom cities by profit
plt.figure(figsize=(14, 5))
sns.barplot(x='Profit', y='Country', data=Country_profit.head(10), palette='viri
plt.title('Top 10 Cities by Total Profit')
plt.show()

plt.figure(figsize=(14, 5))
```

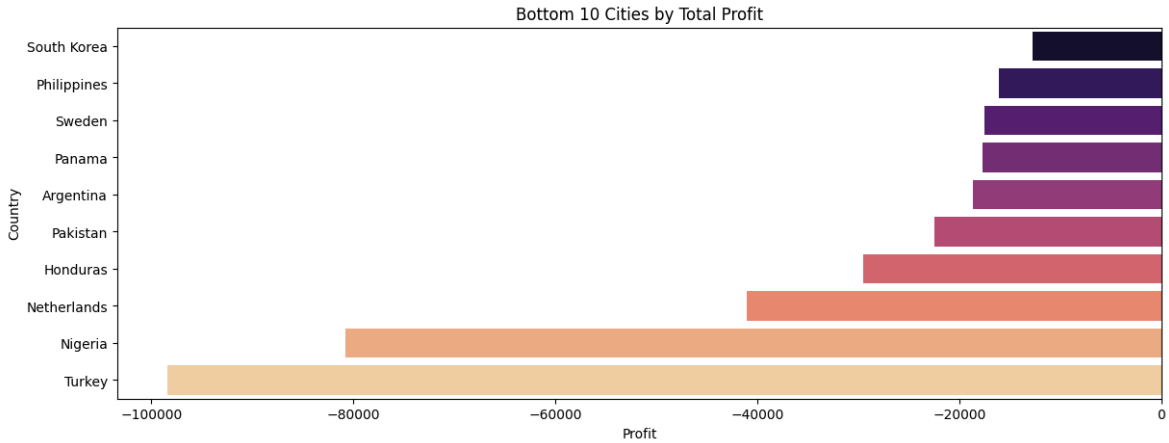
```
sns.barplot(x='Profit', y='Country', data=Country_profit.tail(10), palette='magm
plt.title('Bottom 10 Cities by Total Profit')
```

```
/tmp/ipykernel_556/4266622529.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='Country', data=Country_profit.head(10), palette='vir
idis')
```




```
/tmp/ipykernel_556/4266622529.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='Country', data=Country_profit.tail(10), palette='mag
ma')
```

Out[ ]: Text(0.5, 1.0, 'Bottom 10 Cities by Total Profit')



In [ ]:
```
city_profit
```

Out[ ]:

	City	Profit
2290	New York City	62036.98370
1910	Los Angeles	30440.75790
2936	Seattle	29156.09670
1989	Managua	17853.71804
2843	San Francisco	17507.38540
...	...	...
1760	Lahore	-13626.37200
2499	Philadelphia	-13837.76740
3178	Tegucigalpa	-15007.41608
1482	Istanbul	-19960.90800
1757	Lagos	-25922.51100

3636 rows × 2 columns

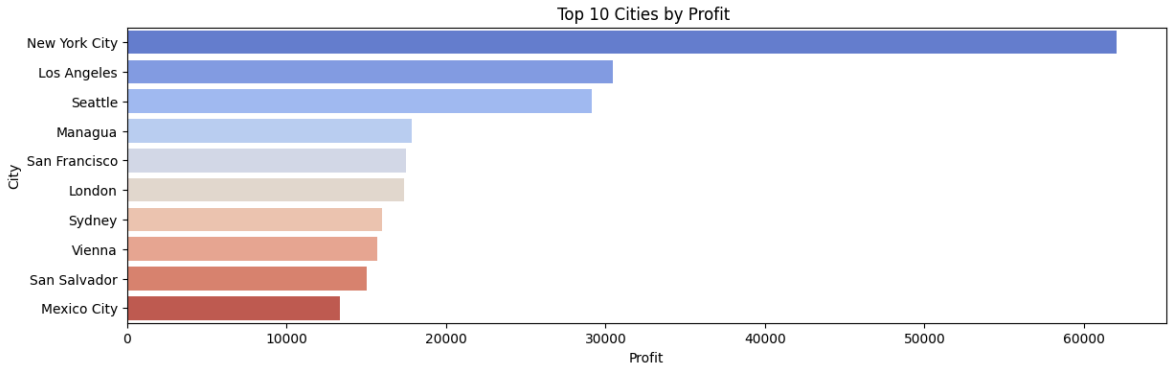
In [ ]:

```
#Profit Margin based on city and region

# ProfMarginit margin for each city
data['Profit ']= (data['Profit'] / data['Sales'])*100
city_profit_margin = data.groupby('City')['Profit'].mean().reset_index().sort_va
```

In [ ]:

```
# Visualize the cities with the highest and lowest profit margins
plt.figure(figsize=(14, 4))
sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='coolwarm')
plt.title('Top 10 Cities by Profit ')
plt.show()
```

```
/tmp/ipykernel_556/353401179.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit.head(10), palette='coolwar
m')
```



In [ ]:

```
plt.figure(figsize=(14, 4))
sns.barplot(x='Profit', y='City', data=city_profit_margin.tail(10), palette='coo
```

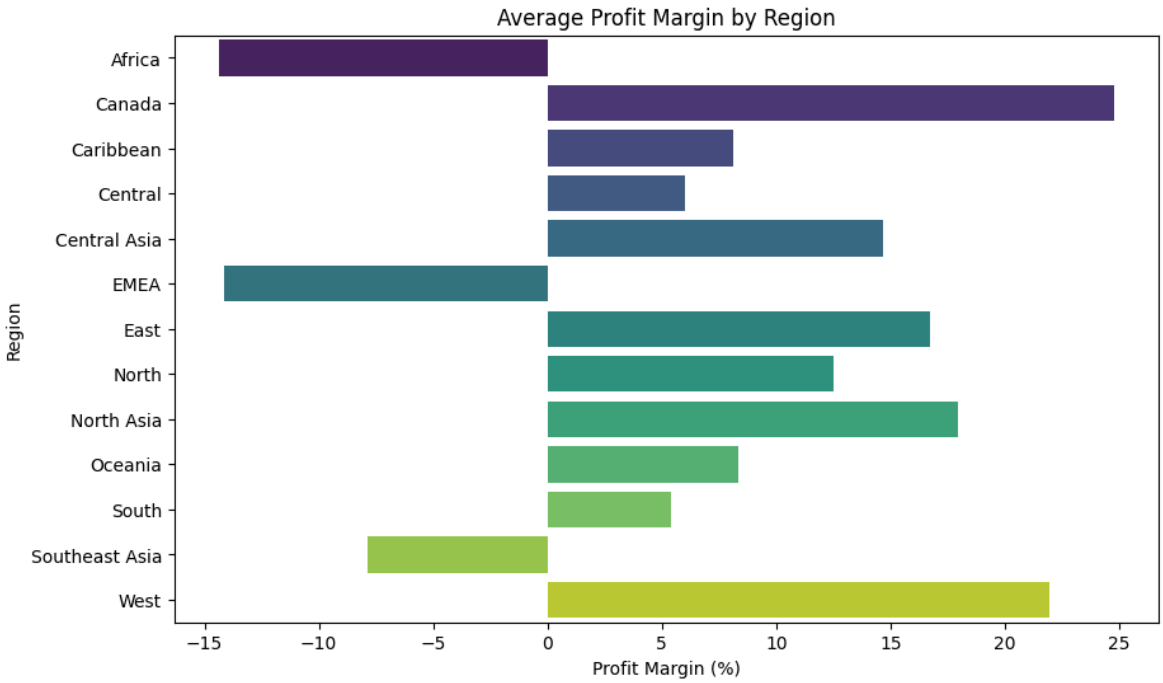
```
plt.title('Bottom 10 Cities by Profit')
plt.show()
```

```
/tmp/ipykernel_556/6070053.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit', y='City', data=city_profit_margin.tail(10), palette='co
olwarm')
```

In [ ]:
```
# Calculate profit margin by region

data['Profit'] = (data['Profit'] / data['Sales']) * 100
Profit = data.groupby('Region')['Profit'].mean().reset_index()
```

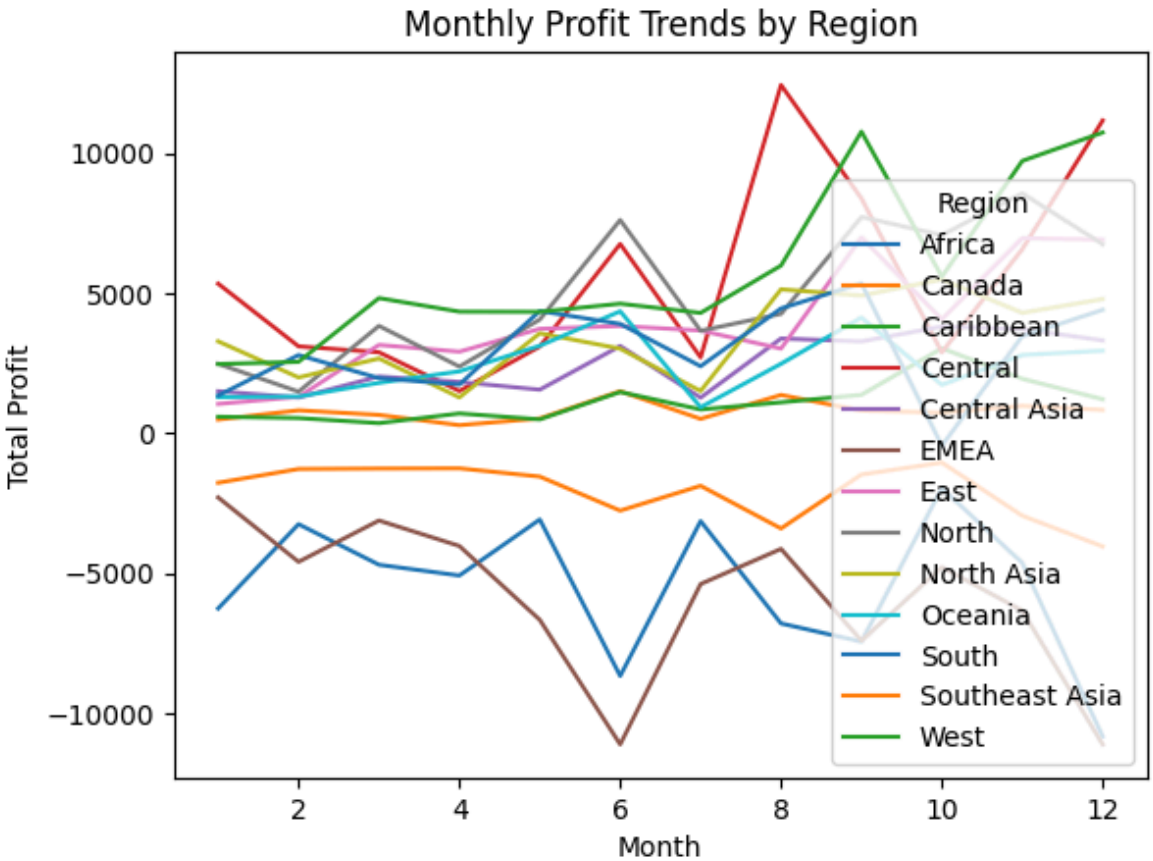
In [ ]:
```
# plotting
plt.figure(figsize=(10, 6))
sns.barplot(x='Profit Margin', y='Region', data=profit_margin_region, palette='v
plt.title('Average Profit Margin by Region')
plt.xlabel('Profit Margin (%)')
plt.ylabel('Region')
plt.show()
```

```
/tmp/ipykernel_556/2235298481.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Profit Margin', y='Region', data=profit_margin_region, palette
='viridis')
```



In [ ]:
```
#Profitability Over Time by Region
monthly_profit = data.groupby(['month', 'Region'])['Profit'].sum().unstack().fil
# plotting
plt.figure(figsize=(25, 7))
monthly_profit.plot(kind='line')
plt.title('Monthly Profit Trends by Region')
plt.ylabel('Total Profit')
plt.xlabel('Month')
plt.show()
```

```
<Figure size 2500x700 with 0 Axes>
```



In [ ]:
```
monthly_profit['Total Profit'] = monthly_profit.sum(axis=1)

# Sort the DataFrame by 'Total Profit' in descending order
monthly_profit_sorted = monthly_profit.sort_values(by='Total Profit', ascending=

# Display the top 4 rows
monthly_profit_sorted.head(4)
```

Out[ ]:

Region	Africa	Canada	Caribbean	Central	Central Asia	EM
month						
9	-7449.693941	806.078608	1372.009720	8390.381399	3280.833534	-7415.904
11	-4635.519934	976.896011	1944.131502	6576.269679	3720.982205	-6341.397
8	-6798.336985	1367.823287	1103.289191	12443.953421	3384.779031	-4134.467
12	-10842.694811	832.446669	1214.014909	11176.577842	3316.743326	-11123.133

Shipping Cost Analysis

In [ ]:
```
import pandas as pd
import numpy as np

# 🔷 Clean column names (avoid hidden space issues)
data.columns = data.columns.str.strip()

# 🔷 Ensure correct data type
data['Shipping Cost'] = pd.to_numeric(data['Shipping Cost'], errors='coerce')

# 🔷 Group and aggregate
city_shipping = data.groupby('City', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

# 🔷 Get top and bottom cities (Total Shipping Cost)
top_city_total = city_shipping.loc[city_shipping['Total_Shipping_Cost'].idxmax()
bottom_city_total = city_shipping.loc[city_shipping['Total_Shipping_Cost'].idxmi

# 🔷 Get top and bottom cities (Average Shipping Cost)
top_city_avg = city_shipping.loc[city_shipping['Average_Shipping_Cost'].idxmax()
bottom_city_avg = city_shipping.loc[city_shipping['Average_Shipping_Cost'].idxmi

# 🔷 Overall average
average_city_shipping_cost = city_shipping['Average_Shipping_Cost'].mean()

# 🔷 Create summary table
summary_df = pd.DataFrame({
    'Metric': [
        'Top City by Total Shipping Cost',
        'Bottom City by Total Shipping Cost',
        'Top City by Average Shipping Cost',
        'Bottom City by Average Shipping Cost',
        'Average Shipping Cost Across All Cities'
    ],
```

```
    'City': [
        top_city_total['City'],
        bottom_city_total['City'],
        top_city_avg['City'],
        bottom_city_avg['City'],
        np.nan
    ],
    'Total Shipping Cost': [
        top_city_total['Total_Shipping_Cost'],
        bottom_city_total['Total_Shipping_Cost'],
        np.nan,
        np.nan,
        np.nan
    ],
    'Average Shipping Cost': [
        np.nan,
        np.nan,
        top_city_avg['Average_Shipping_Cost'],
        bottom_city_avg['Average_Shipping_Cost'],
        average_city_shipping_cost
    ]
})

# 🔷 Display result
print(summary_df)
```

```
                                  Metric            City  \
0          Top City by Total Shipping Cost   New York City
1       Bottom City by Total Shipping Cost  Victoria Falls
2        Top City by Average Shipping Cost          Kamina
3     Bottom City by Average Shipping Cost  Victoria Falls
4  Average Shipping Cost Across All Cities             NaN

   Total Shipping Cost  Average Shipping Cost
0             26948.17                    NaN
1                 0.09                    NaN
2                  NaN             678.150000
3                  NaN               0.090000
4                  NaN              27.656888
```

In [ ]:
```
# calculate totaland average shipping cost by region
region_shipping = data.groupby('Region')['Shipping Cost'].agg(['sum', 'mean']).r
region_shipping.columns = ['Region', 'Total Shipping Cost', 'Average Shipping Co
```

In [ ]:
```
# Get top and bottom regions by total shipping cost
top_region_total = region_shipping.loc[region_shipping['Total Shipping Cost'].id
```

In [ ]:
```
# Get top and bottom regions by average shipping cost
top_region_total = region_shipping.loc[region_shipping['Average Shipping Cost'].
bottom_region_avg = region_shipping.loc[region_shipping['Average Shipping Cost']
```

In [ ]:
```
# Calculate average shipping cost across all regions
average_region_shipping_cost = region_shipping['Average Shipping Cost'].mean()
```

In [ ]:
```
# Create a concise summary DataFrame
summary_region_df=pd.DataFrame({
    'metric':[
        'Top Region by Total Shipping Cost',
        'Bottom Region by Total Shipping Cost',
```

```
        'Top Region by Average Shipping Cost',
        'Bottom Region by Average Shipping Cost',
        'Average Shipping Cost Across All Regions'
    ],
    'Region':[
        top_region_total['Region'],
        bottom_region_total['Region'],
        top_region_avg['Region'],
        bottom_region_avg['Region'],
      'N/A'
    ],
    'Total Shipping Cost':[
        top_region_total['Total Shipping Cost'],
        bottom_region_total['Total Shipping Cost'],


})
```

In [ ]:

```
import numpy as np

import pandas as pd
import numpy as np

#  Clean column names
data.columns = data.columns.str.strip()

#  Ensure numeric type
data['Shipping Cost'] = pd.to_numeric(data['Shipping Cost'], errors='coerce')

#  Group by Region
region_shipping = data.groupby('Region', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

#  Top & Bottom Regions (Total Shipping Cost)
top_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost'].id
bottom_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost']

#  Top & Bottom Regions (Average Shipping Cost)
top_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost'].id
bottom_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost']

#  Overall average
average_region_shipping_cost = region_shipping['Average_Shipping_Cost'].mean()

#  Create summary DataFrame
summary_region_df = pd.DataFrame({
    'Metric': [
        'Top Region by Total Shipping Cost',
        'Bottom Region by Total Shipping Cost',
        'Top Region by Average Shipping Cost',
        'Bottom Region by Average Shipping Cost',
        'Average Shipping Cost Across All Regions'
    ],
    'Region': [
        top_region_total['Region'],
        bottom_region_total['Region'],
        top_region_avg['Region'],
        bottom_region_avg['Region'],
```

```
        np.nan
    ],
    'Total_Shipping_Cost': [
        top_region_total['Total_Shipping_Cost'],
        bottom_region_total['Total_Shipping_Cost'],
        np.nan,
        np.nan,
        np.nan
    ],
    'Average_Shipping_Cost': [
        np.nan,
        np.nan,
        top_region_avg['Average_Shipping_Cost'],
        bottom_region_avg['Average_Shipping_Cost'],
        average_region_shipping_cost
    ]
})

# 🔷 Display result
print(summary_region_df)
```

```
                                   Metric      Region  Total_Shipping_Cost  \
0         Top Region by Total Shipping Cost     Central            296940.56
1      Bottom Region by Total Shipping Cost      Canada              7405.63
2       Top Region by Average Shipping Cost  North Asia                  NaN
3    Bottom Region by Average Shipping Cost        EMEA                  NaN
4  Average Shipping Cost Across All Regions         NaN                  NaN

   Average_Shipping_Cost
0                    NaN
1                    NaN
2              40.646112
3              17.573221
4              26.867990
```

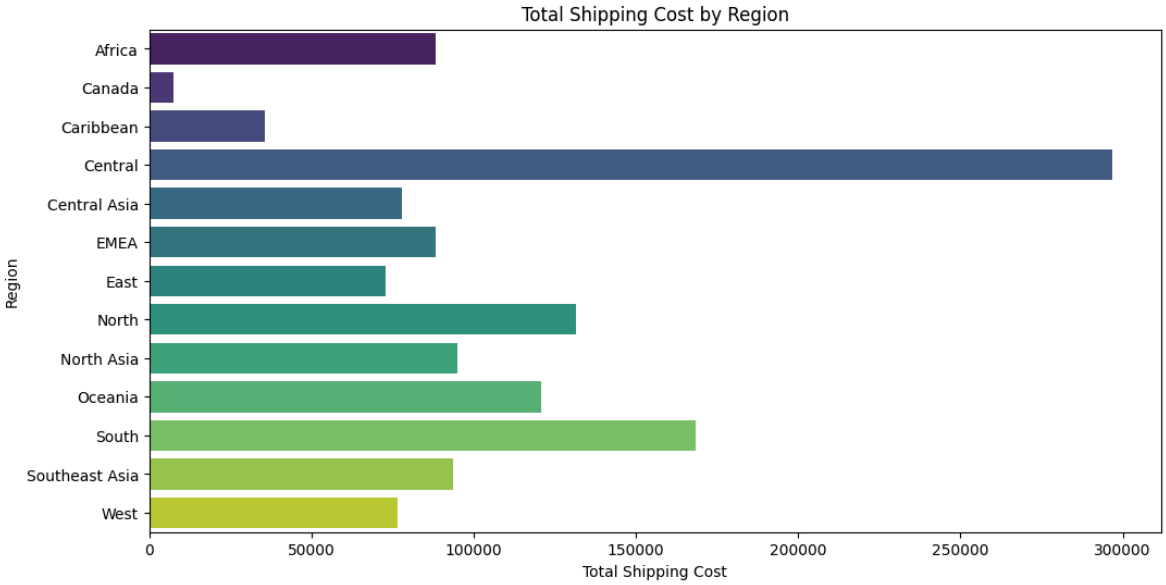
In [ ]:
```
# Group by Region
region_shipping = data.groupby('Region', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

#  Top & Bottom (Total Shipping Cost)
top_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost'].id
bottom_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost']

#  Top & Bottom (Average Shipping Cost)
top_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost'].id
bottom_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost']

#  Overall average
average_region_shipping_cost = region_shipping['Average_Shipping_Cost'].mean()
```

In [ ]:
```
# Calculate total and average shipping cost by region
region_shipping = data.groupby('Region')['Shipping Cost'].agg(['sum','mean']).re
region_shipping.columns = ['Region', 'Total Shipping Cost', 'Average Shipping Co
```

In [ ]:
```
# Plot total shipping cost by region
plt.figure(figsize=(12, 6))
sns.barplot(x='Total Shipping Cost', y='Region', data=region_shipping, palette='
plt.title('Total Shipping Cost by Region')
```

```
plt.xlabel('Total Shipping Cost')
plt.ylabel('Region')
plt.show()
```

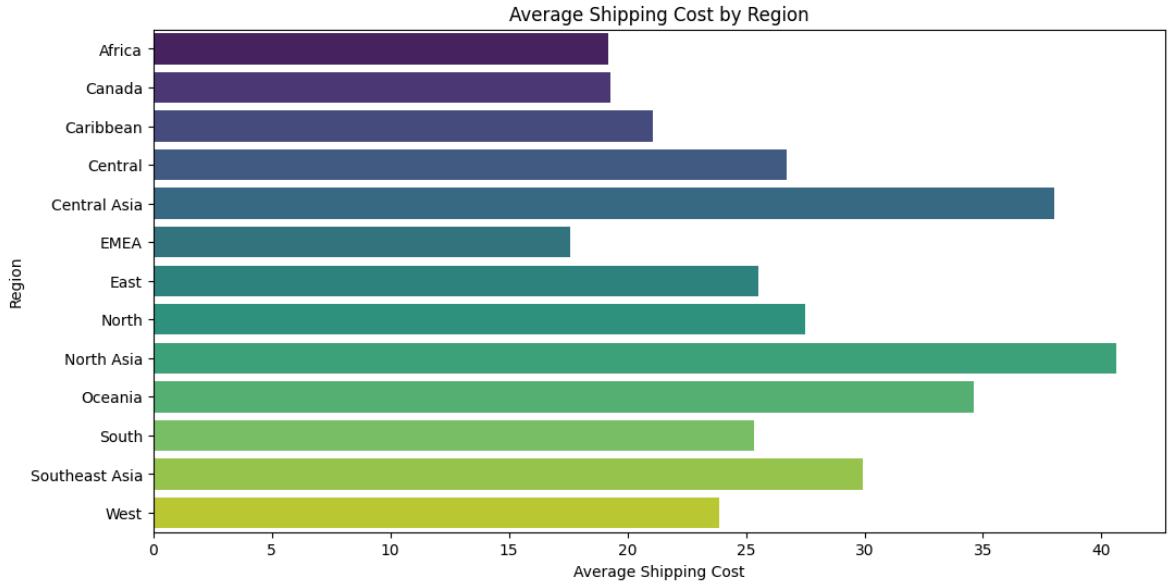
```
/tmp/ipykernel_556/2448218311.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Total Shipping Cost', y='Region', data=region_shipping, palette
='viridis')
```



In [ ]:
```
# Plot average shipping cost by region
plt.figure(figsize=(12, 6))
sns.barplot(x='Average Shipping Cost', y='Region', data=region_shipping, palette
plt.title('Average Shipping Cost by Region')
```

```
/tmp/ipykernel_556/137073443.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Average Shipping Cost', y='Region', data=region_shipping, palett
e='viridis')
```

Out[ ]:
```
Text(0.5, 1.0, 'Average Shipping Cost by Region')
```



In [ ]:
```
# Number of customers by city
City_customer_density = data.groupby('City')['Customer ID'].nunique().reset_inde
City_customer_density.columns = ['City',  'Number of Customers']
```

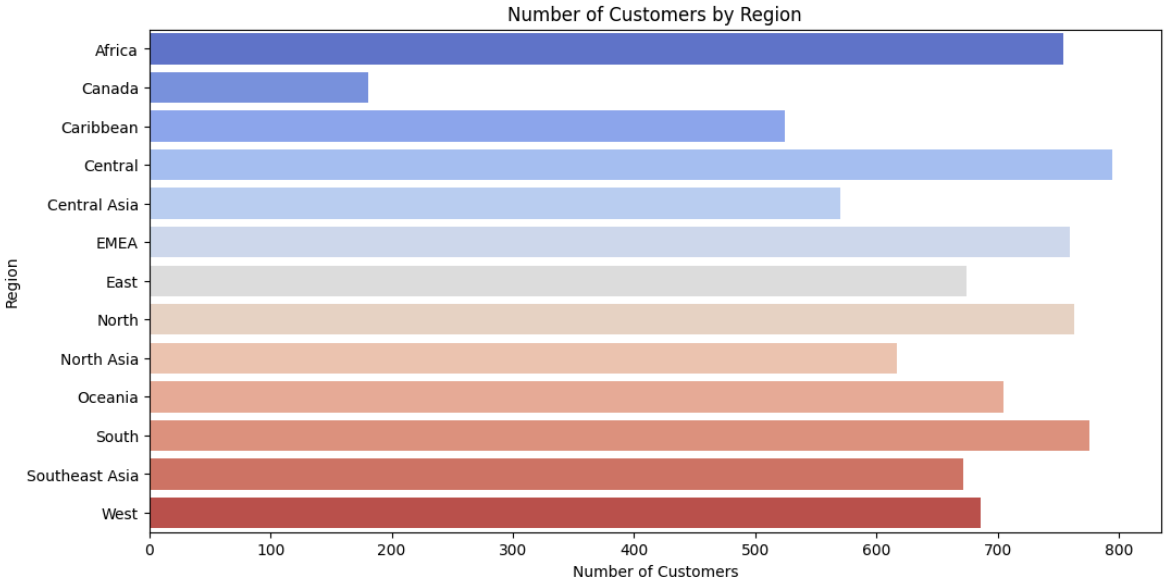
In [ ]:
```
# Number of customers by State
state_customer_density = data.groupby('State')['Customer ID'].nunique().reset_in
state_customer_density.columns = ['State', 'Number of Customers']
```

In [ ]:
```
# Number of customers by Country
country_customer_density = data.groupby('Country')['Customer ID'].nunique().rese
country_customer_density.columns = ['Country', 'Number of Customers']
```

In [ ]:
```
# Number of customers by Region
region_customer_density = data.groupby('Region')['Customer ID'].nunique().reset_
region_customer_density.columns = ['Region', 'Number of Customers']
```

In [ ]:
```
# Plot number of customers by Region
plt.figure(figsize=(12, 6))
sns.barplot(x='Number of Customers', y='Region', data=region_customer_density, p
plt.title('Number of Customers by Region')
plt.xlabel('Number of Customers')
plt.ylabel('Region')
plt.show()
```

```
/tmp/ipykernel_556/3253066131.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Number of Customers', y='Region', data=region_customer_density,
palette='coolwarm')
```



In [ ]:
```
City_customer_density
```

Out[ ]:

	City	Number of Customers
0	Aachen	10
1	Aalen	1
2	Aalst	1
3	Aba	9
4	Abadan	6
...	...	...
3631	Zwedru	1
3632	Zwickau	3
3633	Zwolle	2
3634	eMbalenhle	1
3635	Águas Lindas de Goiás	3

3636 rows × 2 columns

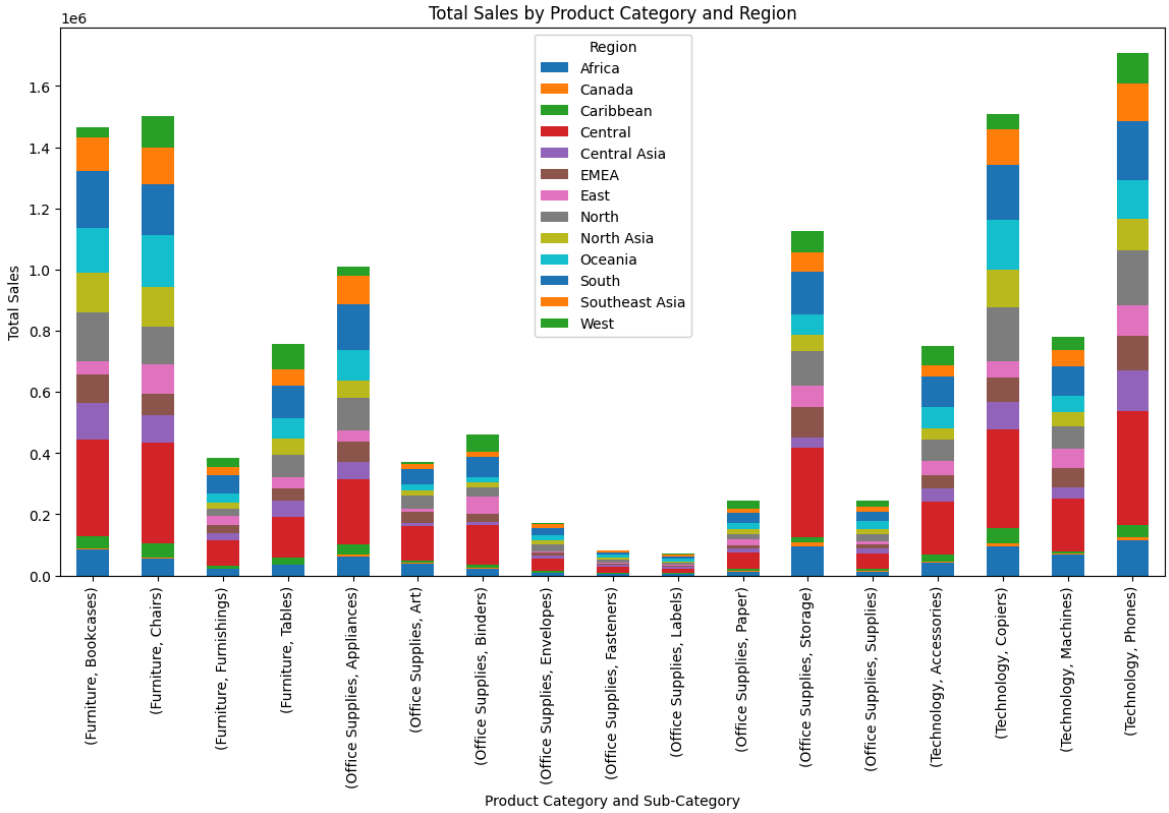
Product Category Analysis by Region

In [ ]:
```
# Group by Category, Sub-Category, and Region

category_region_sales = data.groupby(['Category', 'Sub-Category', 'Region'])['Sa
category_region_profit = data.groupby(['Category', 'Sub-Category', 'Region'])['P
```

In [ ]:
```
# Pivot tables for easier plotting
sales_pivot = category_region_sales.pivot_table(index=['Category', 'Sub-Category
profit_pivot = category_region_profit.pivot_table(index=['Category', 'Sub-Catego
```

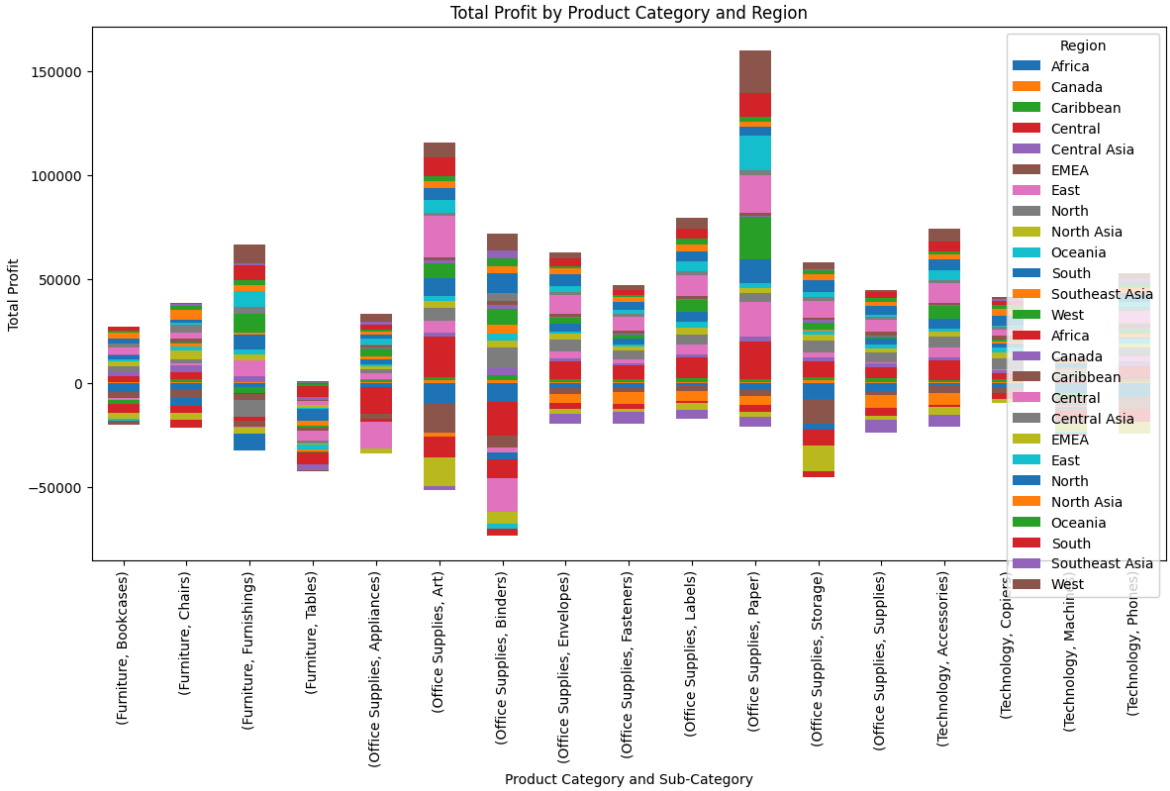
In [ ]:
```
# Plotting Sales by Region
plt.figure(figsize=(14, 7))
sales_pivot.plot(kind='bar', stacked=True, figsize=(14, 7))
plt.title('Total Sales by Product Category and Region')
plt.xlabel('Product Category and Sub-Category')
plt.ylabel('Total Sales')
plt.legend(title='Region')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



In [ ]:
```
# Plotting Profit by Region
plt.figure(figsize=(14, 7))
profit_pivot.plot(kind='bar', stacked=True, figsize=(14, 7))
plt.title('Total Profit by Product Category and Region')
plt.xlabel('Product Category and Sub-Category')
plt.ylabel('Total Profit')
plt.legend(title='Region')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```



In [ ]:
```
sales_percentage = category_region_sales.copy()
sales_percentage = sales_percentage.sort_values(by='Sales', ascending = False)
sales_percentage
```

Out[ ]:

	Category	Sub-Category	Region	Sales
211	Technology	Phones	Central	370198.04400
16	Furniture	Chairs	Central	328571.43700
185	Technology	Copiers	Central	322961.87864
3	Furniture	Bookcases	Central	315419.41480
146	Office Supplies	Storage	Central	292402.30900
...	...	...	...	...
110	Office Supplies	Fasteners	East	819.71800
27	Furniture	Furnishings	Canada	808.53000
92	Office Supplies	Envelopes	Canada	567.18000
105	Office Supplies	Fasteners	Canada	479.46000
118	Office Supplies	Labels	Canada	421.74000

221 rows × 4 columns

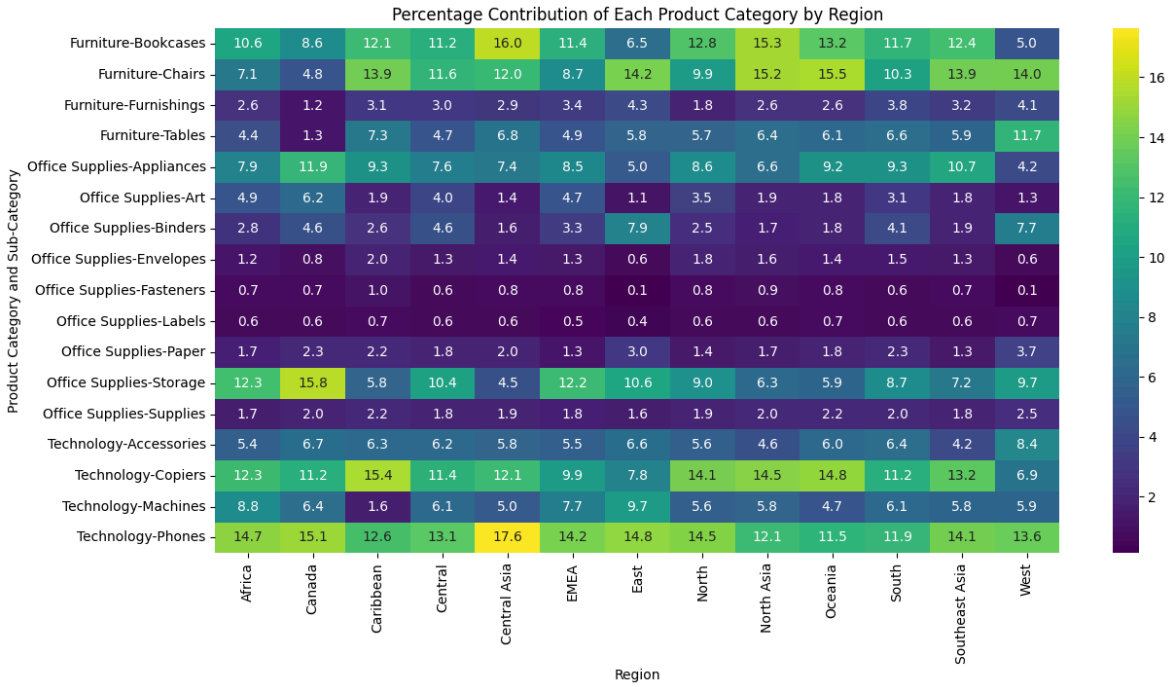
In [ ]:
```
#Calculate percentage contribution based on region and sales
total_sales_by_region = sales_percentage.groupby('Region')['Sales'].sum().reset_
total_sales_by_region.columns = ['Region', 'Total Sales']
```

In [ ]:
```
# Merge total sales back into the main DataFrame
```

```
sales_percentage = sales_percentage.merge(total_sales_by_region, on='Region')
```

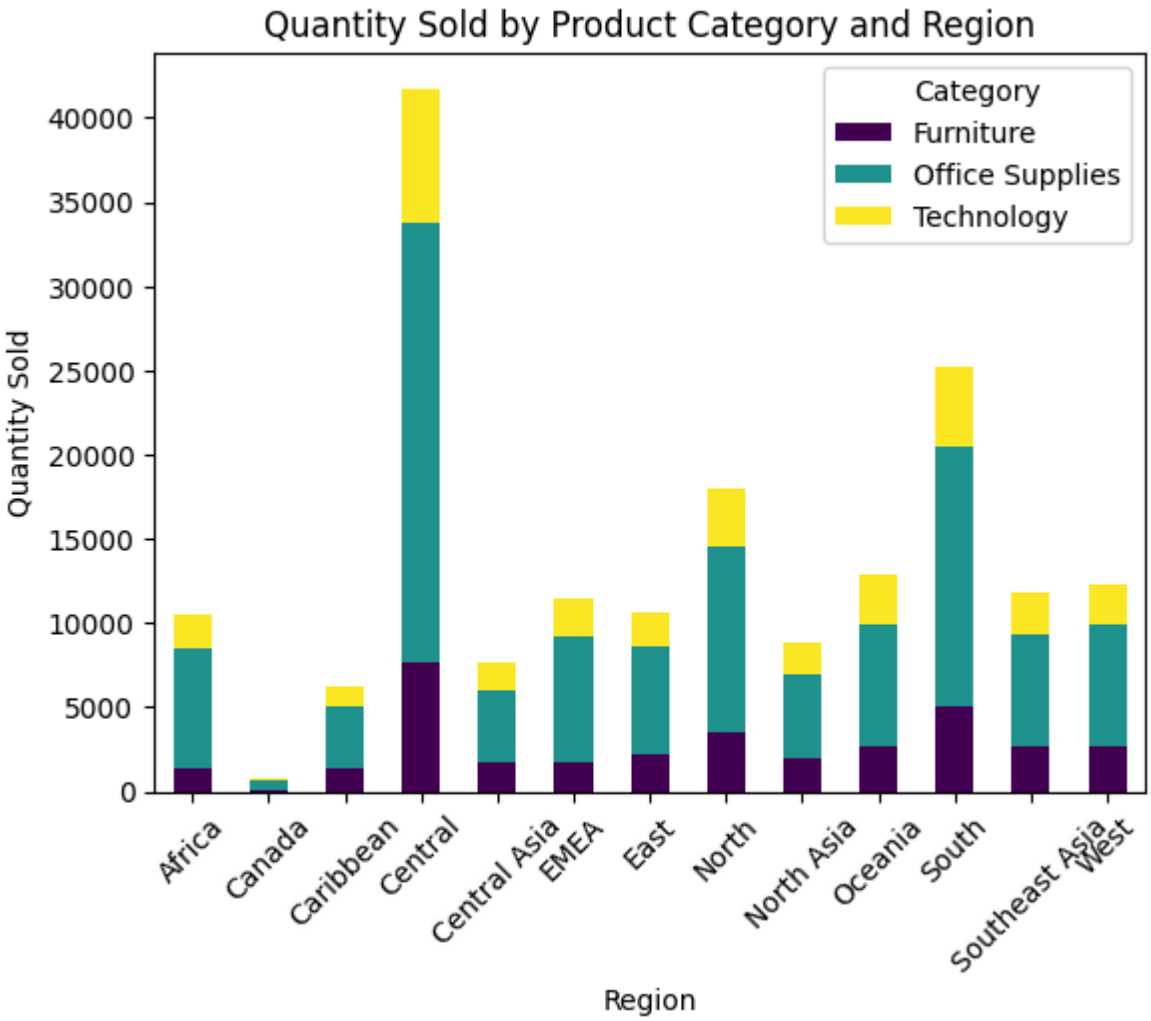
In [ ]:
```
# Calculate percentage contribution
sales_percentage['Percentage'] = (sales_percentage['Sales'] / sales_percentage['
sales_percentage_pivot = sales_percentage.pivot_table(index=['Category', 'Sub-Ca
```

In [ ]:
```
# Plotting Percentage Contribution
plt.figure(figsize=(14, 7))
sns.heatmap(sales_percentage_pivot, cmap='viridis', annot=True, fmt=".1f")
plt.title('Percentage Contribution of Each Product Category by Region')
plt.xlabel('Region')
plt.ylabel('Product Category and Sub-Category')
plt.show()
```



In [ ]:
```
#Quantity Sold by Product Category and Region
quantity_sold_by_region = data.groupby(['Region', 'Category'])['Quantity'].sum()
quantity_sold_by_region = quantity_sold_by_region.pivot(index='Region', columns=

# Plot Quantity Sold by Product Category and Region
plt.figure(figsize=(14, 7))
quantity_sold_by_region.plot(kind='bar', stacked=True, colormap='viridis')
plt.title('Quantity Sold by Product Category and Region')
plt.xlabel('Region')
plt.ylabel('Quantity Sold')
plt.xticks(rotation=45)
plt.legend(title='Category')
plt.show()
```

```
<Figure size 1400x700 with 0 Axes>
```

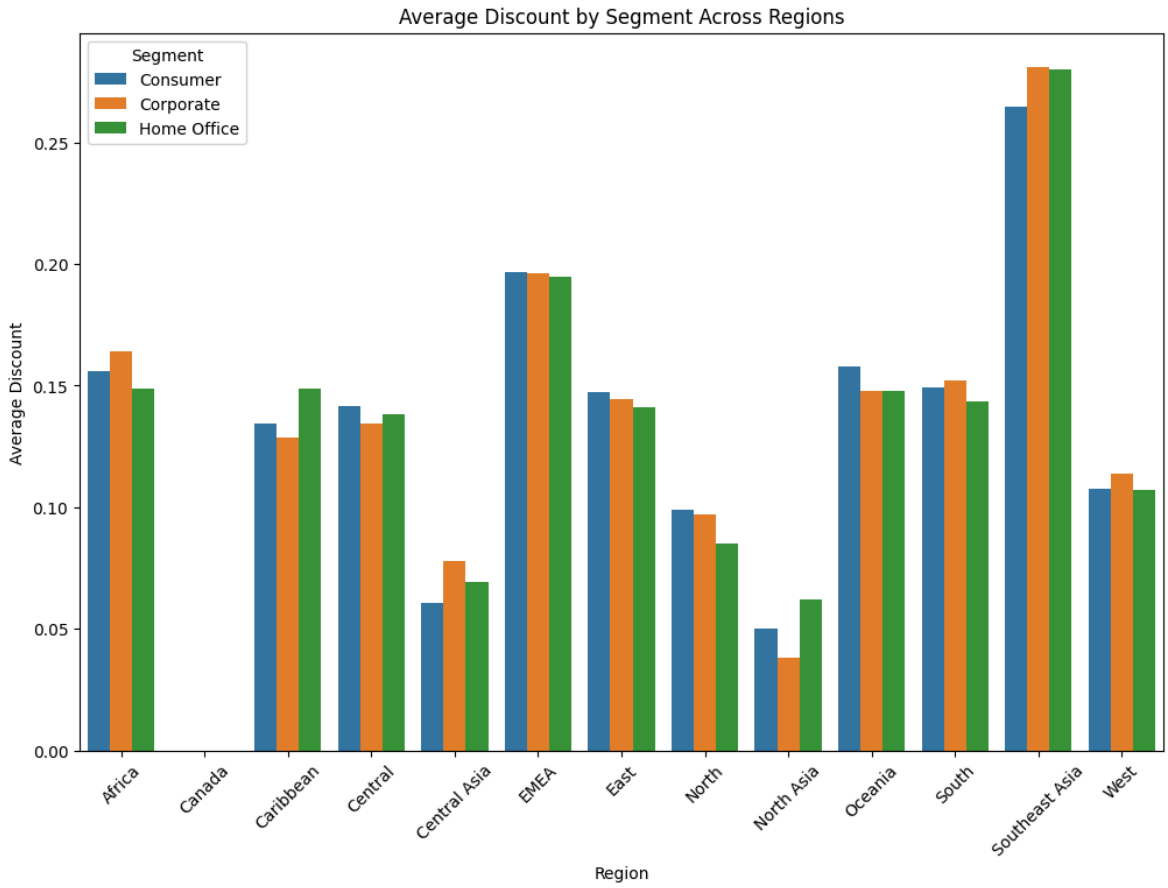


In [ ]:
```
# Remove duplicate columns
data = data.loc[:, ~data.columns.duplicated()]

# Clean column names
data.columns = data.columns.str.strip()

# Convert to numeric
data['Profit'] = pd.to_numeric(data['Profit'], errors='coerce')
```

```
/tmp/ipykernel_556/3137141608.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['Profit'] = pd.to_numeric(data['Profit'], errors='coerce')
```

In [ ]:
```
#Average Discount Given by Segment Across Regions:

avg_discount_by_segment_region = data.groupby(['Segment', 'Region'])['Discount']
avg_discount_by_segment_region.columns = ['Segment', 'Region', 'Average Discount

# Plotting the average discount by segment and region
plt.figure(figsize=(12, 8))
sns.barplot(x='Region', y='Average Discount', hue='Segment', data=avg_discount_b
plt.title('Average Discount by Segment Across Regions')
plt.xlabel('Region')
plt.ylabel('Average Discount')
```

```
plt.legend(title='Segment')
plt.xticks(rotation=45)
plt.show()
```



In [ ]:
```
avg_discount_by_segment_region
```

Out[ ]:

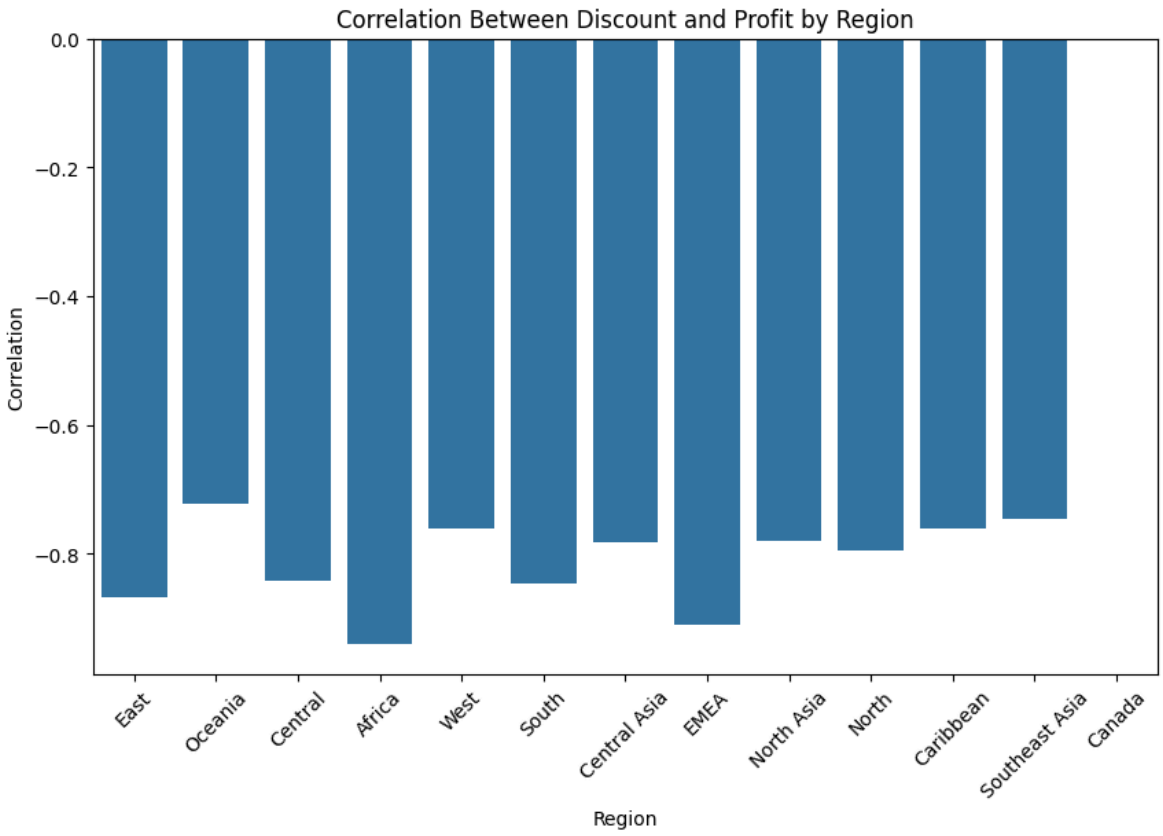
	Segment	Region	Average Discount
0	Consumer	Africa	0.155733
1	Consumer	Canada	0.000000
2	Consumer	Caribbean	0.134435
3	Consumer	Central	0.141548
4	Consumer	Central Asia	0.060749
5	Consumer	EMEA	0.196690
6	Consumer	East	0.147447
7	Consumer	North	0.099059
8	Consumer	North Asia	0.050000
9	Consumer	Oceania	0.157975
10	Consumer	South	0.149212
11	Consumer	Southeast Asia	0.264461
12	Consumer	West	0.107506
13	Corporate	Africa	0.163948
14	Corporate	Canada	0.000000
15	Corporate	Caribbean	0.128746
16	Corporate	Central	0.134564
17	Corporate	Central Asia	0.077651
18	Corporate	EMEA	0.195934
19	Corporate	East	0.144356
20	Corporate	North	0.097243
21	Corporate	North Asia	0.037994
22	Corporate	Oceania	0.147768
23	Corporate	South	0.151928
24	Corporate	Southeast Asia	0.280935
25	Corporate	West	0.113958
26	Home Office	Africa	0.148658
27	Home Office	Canada	0.000000
28	Home Office	Caribbean	0.148828
29	Home Office	Central	0.138178
30	Home Office	Central Asia	0.069466
31	Home Office	EMEA	0.194656
32	Home Office	East	0.141036

	Segment	Region	Average Discount
33	Home Office	North	0.085002
34	Home Office	North Asia	0.061957
35	Home Office	Oceania	0.147906
36	Home Office	South	0.143644
37	Home Office	Southeast Asia	0.280000
38	Home Office	West	0.106918

In [ ]:

```
# Create an empty dictionary to store the correlations
correlation_by_region = {}

# Loop through each region and calculate the correlation
for region in data['Region'].unique():
    # Filter the data for the current region
    region_data = data[data['Region'] == region]

    # Calculate the correlation between Discount and Profit
    correlation = region_data['Discount'].corr(region_data['Profit'])

    # Store the result in the dictionary
    correlation_by_region[region] = correlation

# Convert the dictionary to a DataFrame
correlation_by_region_df = pd.DataFrame(list(correlation_by_region.items()), col
correlation_by_region_df
plt.figure(figsize=(10, 6))
sns.barplot(x='Region', y='Discount-Profit Correlation', data=correlation_by_reg
plt.title('Correlation Between Discount and Profit by Region')
plt.xlabel('Region')
plt.ylabel('Correlation')
plt.xticks(rotation=45)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2922: Ru
ntimeWarning: invalid value encountered in divide
  c /= stddev[:, None]
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.py:2923: Ru
ntimeWarning: invalid value encountered in divide
  c /= stddev[None, :]
```



In [ ]:
```
data.head(3)
```

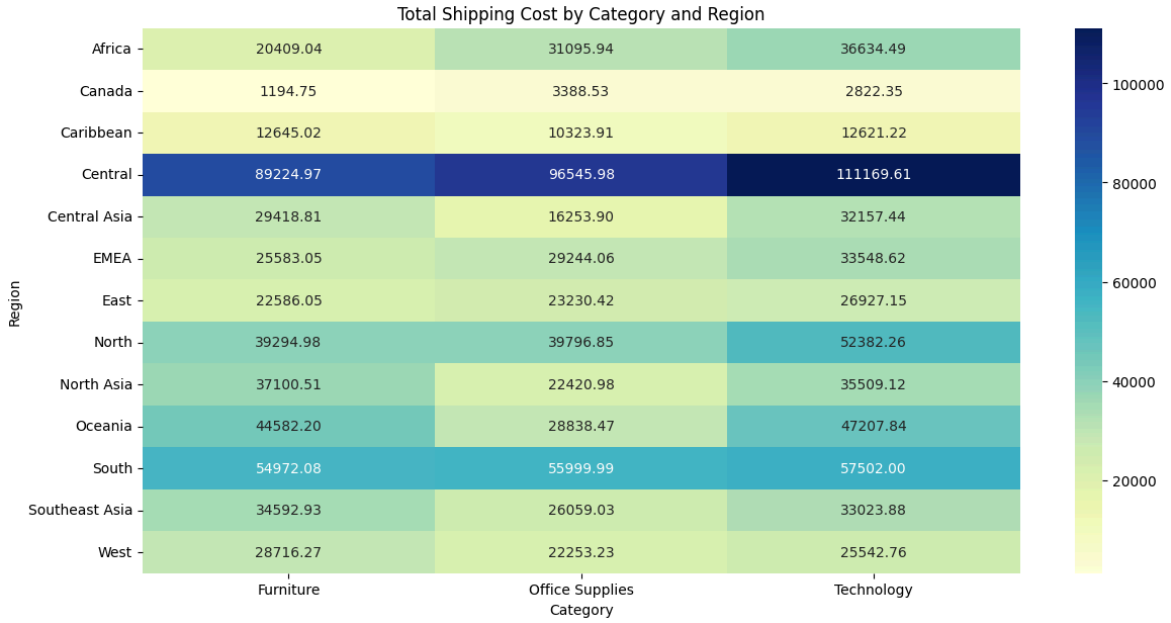
Out[ ]:

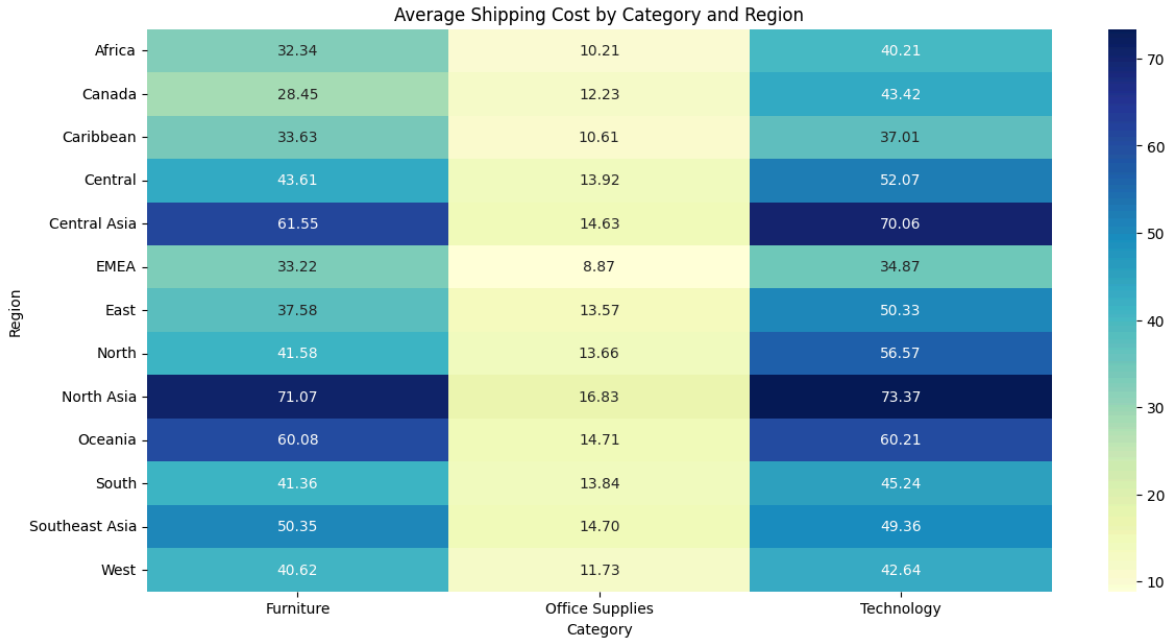
	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	2013-02-05	2013-02-07	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	2013-10-17	2013-10-18	First Class	CR-12730	Craig Reiter	Consumer	Brisbane

3 rows × 26 columns

In [ ]:
```
#Examine the relationship between sales and shipping costs across regions.
# Total and Average Shipping Cost by Category and Region
shipping_by_cat_reg = data.groupby(['Category', 'Region'])['Shipping Cost'].agg(
shipping_by_cat_reg.columns = ['Category', 'Region', 'Total Shipping Cost', 'Ave

# Pivot tables for visualization
```

```
total_shipping_pivot = shipping_by_cat_reg.pivot(index='Region', columns='Catego
average_shipping_pivot = shipping_by_cat_reg.pivot(index='Region', columns='Cate

# Plot Total Shipping Costs
plt.figure(figsize=(14, 7))
sns.heatmap(total_shipping_pivot, annot=True, cmap='YlGnBu', fmt='.2f')
plt.title('Total Shipping Cost by Category and Region')
plt.xlabel('Category')
plt.ylabel('Region')
plt.show()

# Plot Average Shipping Costs
plt.figure(figsize=(14, 7))
sns.heatmap(average_shipping_pivot, annot=True, cmap='YlGnBu', fmt='.2f')
plt.title('Average Shipping Cost by Category and Region')
plt.xlabel('Category')
plt.ylabel('Region')
plt.show()
```





In [ ]:
```
shipping_by_cat_reg
```

Out[ ]:

	Category	Region	Total Shipping Cost	Average Shipping Cost
0	Furniture	Africa	20409.04	32.343962
1	Furniture	Canada	1194.75	28.446429
2	Furniture	Caribbean	12645.02	33.630372
3	Furniture	Central	89224.97	43.609467
4	Furniture	Central Asia	29418.81	61.545628
5	Furniture	EMEA	25583.05	33.224740
6	Furniture	East	22586.05	37.580782
7	Furniture	North	39294.98	41.581989
8	Furniture	North Asia	37100.51	71.073774
9	Furniture	Oceania	44582.20	60.083827
10	Furniture	South	54972.08	41.363491
11	Furniture	Southeast Asia	34592.93	50.353610
12	Furniture	West	28716.27	40.617072
13	Office Supplies	Africa	31095.94	10.212131
14	Office Supplies	Canada	3388.53	12.232960
15	Office Supplies	Caribbean	10323.91	10.610391
16	Office Supplies	Central	96545.98	13.919547
17	Office Supplies	Central Asia	16253.90	14.629973
18	Office Supplies	EMEA	29244.06	8.869900
19	Office Supplies	East	23230.42	13.569171
20	Office Supplies	North	39796.85	13.657121
21	Office Supplies	North Asia	22420.98	16.832568
22	Office Supplies	Oceania	28838.47	14.706002
23	Office Supplies	South	55999.99	13.844250
24	Office Supplies	Southeast Asia	26059.03	14.697704
25	Office Supplies	West	22253.23	11.730749
26	Technology	Africa	36634.49	40.213491
27	Technology	Canada	2822.35	43.420769
28	Technology	Caribbean	12621.22	37.012375
29	Technology	Central	111169.61	52.070075
30	Technology	Central Asia	32157.44	70.059782
31	Technology	EMEA	33548.62	34.873825
32	Technology	East	26927.15	50.331121

	Category	Region	Total Shipping Cost	Average Shipping Cost
33	Technology	North	52382.26	56.568315
34	Technology	North Asia	35509.12	73.365950
35	Technology	Oceania	47207.84	60.214082
36	Technology	South	57502.00	45.241542
37	Technology	Southeast Asia	33023.88	49.363049
38	Technology	West	25542.76	42.642337

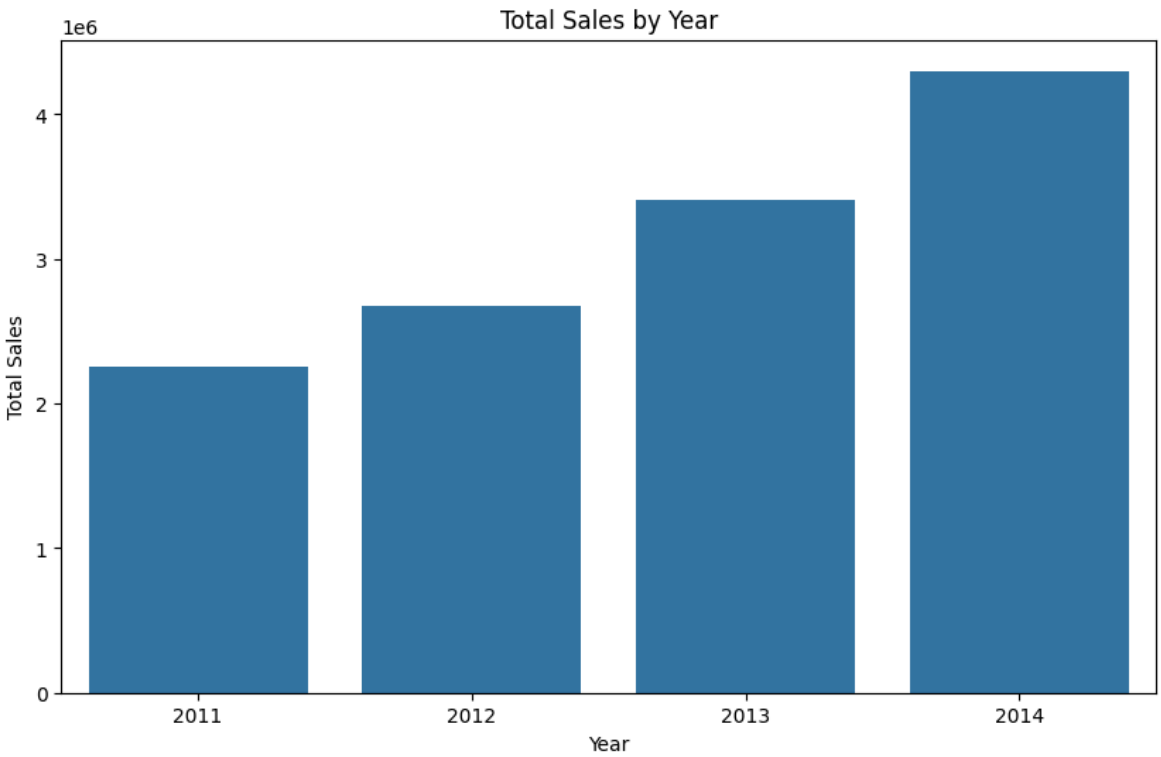
In [ ]:
```
data.head(2)
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	2013-02-05	2013-02-07	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong

2 rows × 26 columns

In [ ]:
```
data['year'] = pd.to_datetime(data['Order Date']).dt.year
data['month'] = pd.to_datetime(data['Order Date']).dt.month
data['day'] = pd.to_datetime(data['Order Date']).dt.day


sales_year = data.groupby('year')['Sales'].sum().reset_index(name= 'Total Sales'
sales_year['YoY Growth (%)'] = sales_year['Total Sales'].pct_change() * 100

sales_year.columns = ['year','Total Sales','Y-o-Y']
sales_year

plt.figure(figsize=(10, 6))
sns.barplot(x='year', y='Total Sales', data=sales_year)
plt.title('Total Sales by Year')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.show()
```



In [ ]:
```
sales_year
```

Out[ ]:

	year	Total Sales	Y-o-Y
0	2011	2.259451e+06	NaN
1	2012	2.677439e+06	18.499530
2	2013	3.405746e+06	27.201659
3	2014	4.299866e+06	26.253259

analyze the time gaps between order dates and shipping dates

In [ ]:
```
data['Time Gap (Days)'] = (data['Ship Date'] - data['Order Date']).dt.days
time_gap_by_ship_mode = data.groupby('Ship Mode')['Time Gap (Days)'].agg(['max',

# Rename columns for clarity
time_gap_by_ship_mode.columns = ['Ship Mode', 'Max Time Gap (day)', 'Average Tim

# Export results to CSV
time_gap_by_ship_mode.to_csv('time_gap_by_ship_mode.csv', index=False)

print("Time gap analysis results have been saved to 'time_gap_by_ship_mode.csv'.
```

```
Time gap analysis results have been saved to 'time_gap_by_ship_mode.csv'.
```

In [ ]:
```
pd.read_csv('time_gap_by_ship_mode.csv')
```

Out[ ]:

	Ship Mode	Max Time Gap (day)	Average Time Gap (day)	Min Time Gap (day)
0	First Class	3	2.181746	1
1	Same Day	1	0.037394	0
2	Second Class	5	3.230187	2
3	Standard Class	7	4.998018	4

EDA ANALYSIS

In [ ]:
```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 29 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Row ID           51290 non-null  int64
 1   Order ID         51290 non-null  object
 2   Order Date       51290 non-null  datetime64[ns]
 3   Ship Date        51290 non-null  datetime64[ns]
 4   Ship Mode        51290 non-null  object
 5   Customer ID      51290 non-null  object
 6   Customer Name    51290 non-null  object
 7   Segment          51290 non-null  object
 8   City             51290 non-null  object
 9   State            51290 non-null  object
 10  Country          51290 non-null  object
 11  Postal Code      51290 non-null  object
 12  Market           51290 non-null  object
 13  Region           51290 non-null  object
 14  Product ID       51290 non-null  object
 15  Category         51290 non-null  object
 16  Sub-Category     51290 non-null  object
 17  Product Name     51290 non-null  object
 18  Sales            51290 non-null  float64
 19  Quantity         51290 non-null  int64
 20  Discount         51290 non-null  float64
 21  Profit           51290 non-null  float64
 22  Shipping Cost    51290 non-null  float64
 23  Order Priority   51290 non-null  object
 24  month            51290 non-null  int32
 25  Profit Margin    51290 non-null  float64
 26  year             51290 non-null  int32
 27  day              51290 non-null  int32
 28  Time Gap (Days)  51290 non-null  int64
dtypes: datetime64[ns](2), float64(5), int32(3), int64(3), object(16)
memory usage: 10.8+ MB
```

In [ ]:

```python
#T-Test: Compare numerical variables (like Sales, Profit) across two groups (e.g
from scipy.stats import ttest_ind

# Group data by Ship Mode
standard_class_sales = data[data['Ship Mode'] == 'Standard Class']['Sales']
first_class_sales = data[data['Ship Mode'] == 'First Class']['Sales']

# Perform the t-test
t_stat, p_val = ttest_ind(standard_class_sales, first_class_sales, equal_var=Fal

print(f"T-statistic: {t_stat}")
print(f"P-value: {p_val}")

# Interpretation
if p_val < 0.05:
    print("There is a significant difference in average sales between Standard C
else:
    print("No significant difference in average sales between Standard Class and
```

```
T-statistic: 0.36742177515410834
P-value: 0.7133111336352411
No significant difference in average sales between Standard Class and First Clas
s.
```

In [ ]:
```
#Chi-Square Test: Check if there's an association between two categorical variab
from scipy.stats import chi2_contingency

# Create a contingency table for Segment and Ship Mode
contingency_table = pd.crosstab(data['Segment'], data['Ship Mode'])

# Perform chi-square test
chi2_stat, p_val, dof, expected = chi2_contingency(contingency_table)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_val}")

# Interpretation
if p_val < 0.05:
    print("There is a significant association between Segment and Ship Mode.")
else:
    print("Segment and Ship Mode are independent.")
```

```
Chi-Square Statistic: 18.274424263215177
P-value: 0.005581770901920851
There is a significant association between Segment and Ship Mode.
```

In [ ]:
```
contingency_table
```

Out[ ]:

Ship Mode Segment	First Class	Same Day	Second Class	Standard Class
Consumer	3918	1467	5257	15876
Corporate	2245	728	3198	9258
Home Office	1342	506	1854	5641

In [ ]:
```
data.head(4)
```

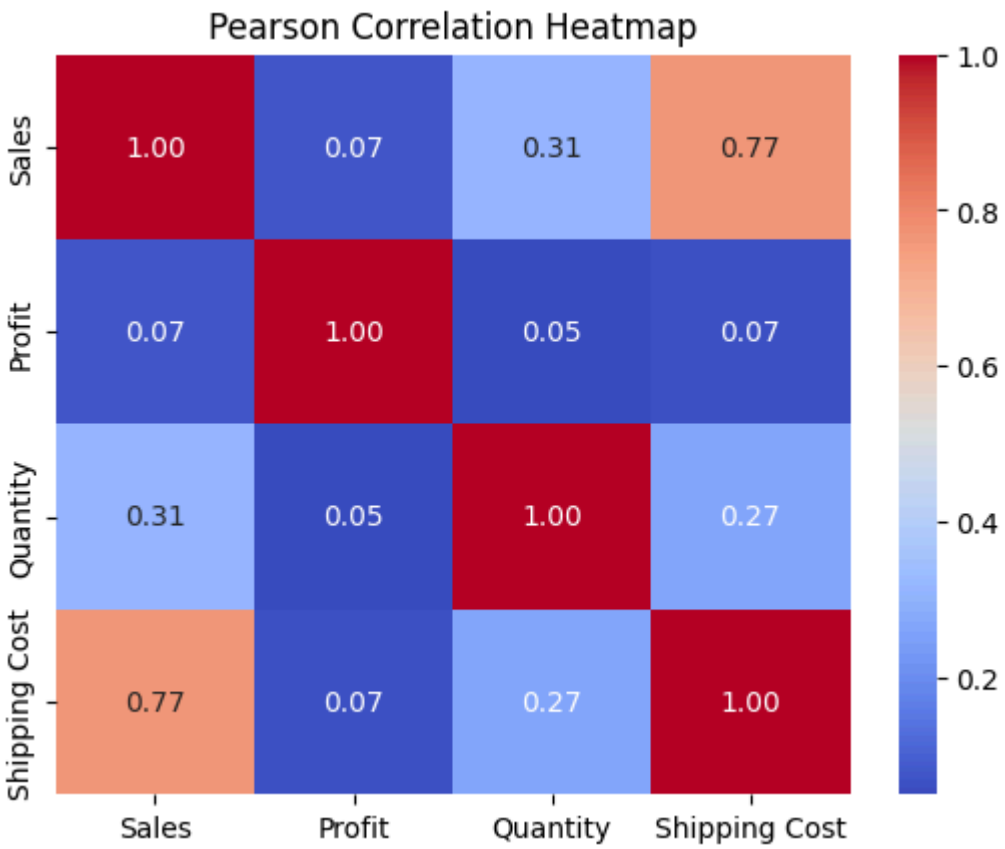
Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	2013-02-05	2013-02-07	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	2013-10-17	2013-10-18	First Class	CR-12730	Craig Reiter	Consumer	Brisbane
3	13524	ES-2013-1579342	2013-01-28	2013-01-30	First Class	KM-16375	Katherine Murray	Home Office	Berlin

4 rows × 29 columns

In [ ]:
```
#Coreleation Analysis

# Pearson Correlation
pearson_corr = data[['Sales', 'Profit', 'Quantity', 'Shipping Cost']].corr(metho
print('Pearson Correlation:\n', pearson_corr)

# Spearman Correlation
spearman_corr = data[['Sales', 'Profit', 'Quantity', 'Shipping Cost']].corr(meth
print('Spearman Correlation:\n', spearman_corr)

# Heatmap
sns.heatmap(pearson_corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Pearson Correlation Heatmap')
plt.show()
```

```
Pearson Correlation:
                   Sales    Profit  Quantity  Shipping Cost
Sales           1.000000  0.074525  0.313577       0.768073
Profit          0.074525  1.000000  0.050143       0.067621
Quantity        0.313577  0.050143  1.000000       0.272649
Shipping Cost   0.768073  0.067621  0.272649       1.000000
Spearman Correlation:
                   Sales    Profit  Quantity  Shipping Cost
Sales           1.000000  0.024350  0.416397       0.913261
Profit          0.024350  1.000000  0.025612       0.025383
Quantity        0.416397  0.025612  1.000000       0.378736
Shipping Cost   0.913261  0.025383  0.378736       1.000000
```



In [ ]:
```
#For Annova testing columns choosen are profit and region
new_data = data[['Profit', 'Region']]
new_data
```

Out[ ]:

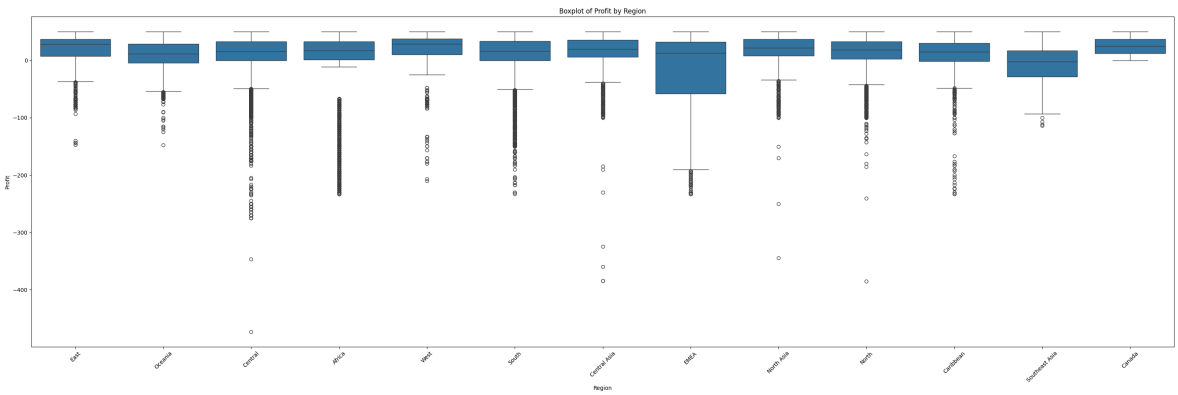
	Profit	Region
0	33.000000	East
1	-7.784693	Oceania
2	17.776630	Oceania
3	-3.337586	Central
4	10.996272	Africa
...	...	...
51285	6.912442	North Asia
51286	-250.000000	Central
51287	49.000000	West
51288	17.857143	South
51289	2.932551	Central

51290 rows × 2 columns

In [ ]:
```
data['Region'].nunique()
```

Out[ ]: 13

In [ ]:
```
plt.figure(figsize=(30, 10))  # Adjust width and height for elongation
sns.boxplot(x='Region', y='Profit', data=new_data)

# Rotate x-axis labels
plt.xticks(rotation=45)

plt.title('Boxplot of Profit by Region')
plt.tight_layout()  # Adjusts layout to prevent clipping of labels
plt.show()
```



In [ ]:
```
np.random.seed(0)  # For reproducibility
random_groups = np.random.choice([f'g{i+1}' for i in range(13)], size=len(data))
new_data['Random Region Group'] = random_groups
```

```
/tmp/ipykernel_556/1205691454.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  new_data['Random Region Group'] = random_groups
```

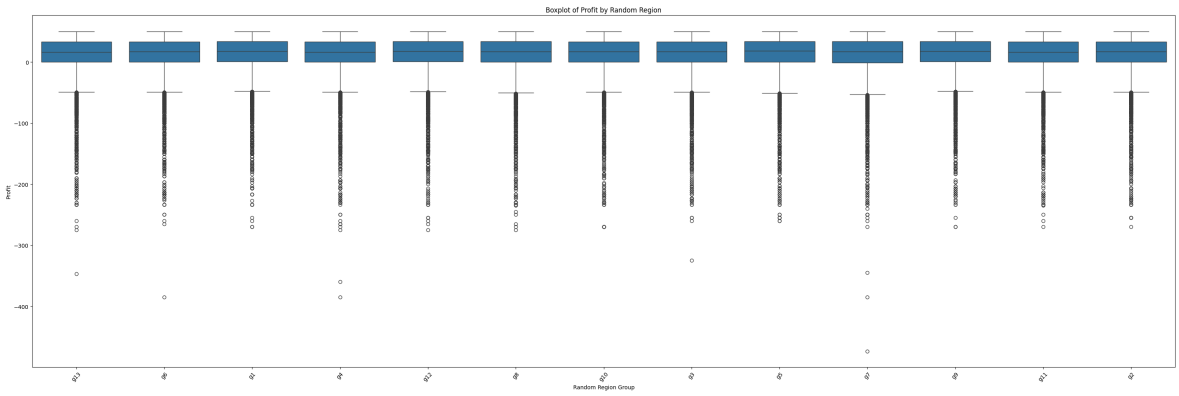
In [ ]:
```
new_data
```

Out[ ]:

	Profit	Region	Random Region Group
0	33.000000	East	g13
1	-7.784693	Oceania	g6
2	17.776630	Oceania	g1
3	-3.337586	Central	g4
4	10.996272	Africa	g12
...	...	...	...
51285	6.912442	North Asia	g6
51286	-250.000000	Central	g7
51287	49.000000	West	g7
51288	17.857143	South	g12
51289	2.932551	Central	g4

51290 rows × 3 columns

In [ ]:
```
plt.figure(figsize=(30, 10))  # Adjust width and height for elongation
sns.boxplot(x='Random Region Group', y='Profit', data=new_data)

# Rotate x-axis labels
plt.xticks(rotation=45)

plt.title('Boxplot of Profit by Random Region')
plt.tight_layout()  # Adjusts layout to prevent clipping of labels
plt.show()
```



In [ ]:
```
new_data['Region'].unique()
```

Out[ ]:
```
array(['East', 'Oceania', 'Central', 'Africa', 'West', 'South',
       'Central Asia', 'EMEA', 'North Asia', 'North', 'Caribbean',
       'Southeast Asia', 'Canada'], dtype=object)
```

In [ ]:
```
new_data['Random Region Group'].unique()
```

Out[ ]:
```
array(['g13', 'g6', 'g1', 'g4', 'g12', 'g8', 'g10', 'g3', 'g5', 'g7',
       'g9', 'g11', 'g2'], dtype=object)
```

In [ ]:
```
# Define profit data for each region
profit_east = new_data[new_data['Region'] == 'East']['Profit']
profit_oceania = new_data[new_data['Region'] == 'Oceania']['Profit']
profit_central = new_data[new_data['Region'] == 'Central']['Profit']
profit_africa = new_data[new_data['Region'] == 'Africa']['Profit']
profit_west = new_data[new_data['Region'] == 'West']['Profit']
profit_south = new_data[new_data['Region'] == 'South']['Profit']
profit_central_asia = new_data[new_data['Region'] == 'Central Asia']['Profit']
profit_emea = new_data[new_data['Region'] == 'EMEA']['Profit']
profit_north_asia = new_data[new_data['Region'] == 'North Asia']['Profit']
profit_north = new_data[new_data['Region'] == 'North']['Profit']
profit_caribbean = new_data[new_data['Region'] == 'Caribbean']['Profit']
profit_southeast_asia = new_data[new_data['Region'] == 'Southeast Asia']['Profit
profit_canada = new_data[new_data['Region'] == 'Canada']['Profit']
```

In [ ]:
```
# Perform One-Way ANOVA for Profit vs Region
import pandas as pd
from scipy.stats import f_oneway
f_stats, p_value = f_oneway(
    profit_east, profit_oceania, profit_central, profit_africa, profit_west,
    profit_south, profit_central_asia, profit_emea, profit_north_asia, profit_no
    profit_caribbean, profit_southeast_asia, profit_canada
)
print("One-Way ANOVA for Profit vs Region:")
print("Test statistic:", f_stats)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one region has a different mean profit.")
else:
    print("Fail to reject H0: All regions have the same mean profit.")
```

```
One-Way ANOVA for Profit vs Region:
Test statistic: 263.5232935059057
P-value: 0.0
Reject H0: At least one region has a different mean profit.
```

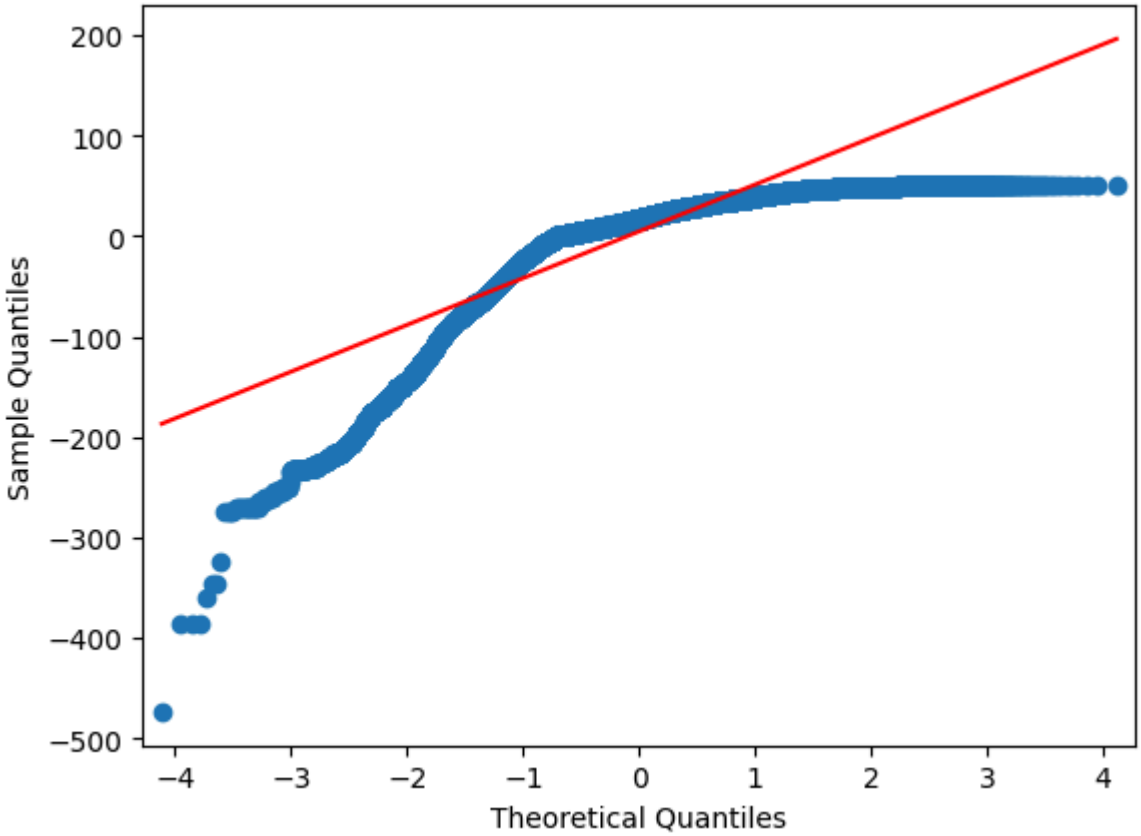
In [ ]:
```
profit_g1 = new_data[new_data['Random Region Group'] == 'g1']['Profit']
profit_g2 = new_data[new_data['Random Region Group'] == 'g2']['Profit']
profit_g3 = new_data[new_data['Random Region Group'] == 'g3']['Profit']
profit_g4 = new_data[new_data['Random Region Group'] == 'g4']['Profit']
profit_g5 = new_data[new_data['Random Region Group'] == 'g5']['Profit']
profit_g6 = new_data[new_data['Random Region Group'] == 'g6']['Profit']
profit_g7 = new_data[new_data['Random Region Group'] == 'g7']['Profit']
profit_g8 = new_data[new_data['Random Region Group'] == 'g8']['Profit']
profit_g9 = new_data[new_data['Random Region Group'] == 'g9']['Profit']
profit_g10 = new_data[new_data['Random Region Group'] == 'g10']['Profit']
profit_g11 = new_data[new_data['Random Region Group'] == 'g11']['Profit']
profit_g12 = new_data[new_data['Random Region Group'] == 'g12']['Profit']
profit_g13 = new_data[new_data['Random Region Group'] == 'g13']['Profit']
```

In [ ]:
```
# Perform One-Way ANOVA for Profit vs Random Region Group
f_stats, p_value = f_oneway(
    profit_g1, profit_g2, profit_g3, profit_g4, profit_g5, profit_g6,
    profit_g7, profit_g8, profit_g9, profit_g10, profit_g11, profit_g12, profit_
)

print("\nOne-Way ANOVA for Profit vs Random Region Group:")
print("Test statistic:", f_stats)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one random group has a different mean profit.")
else:
    print("Fail to reject H0: All random groups have the same mean profit.")
```

```
One-Way ANOVA for Profit vs Random Region Group:
Test statistic: 1.6016851568807748
P-value: 0.08339021565889154
Fail to reject H0: All random groups have the same mean profit.
```

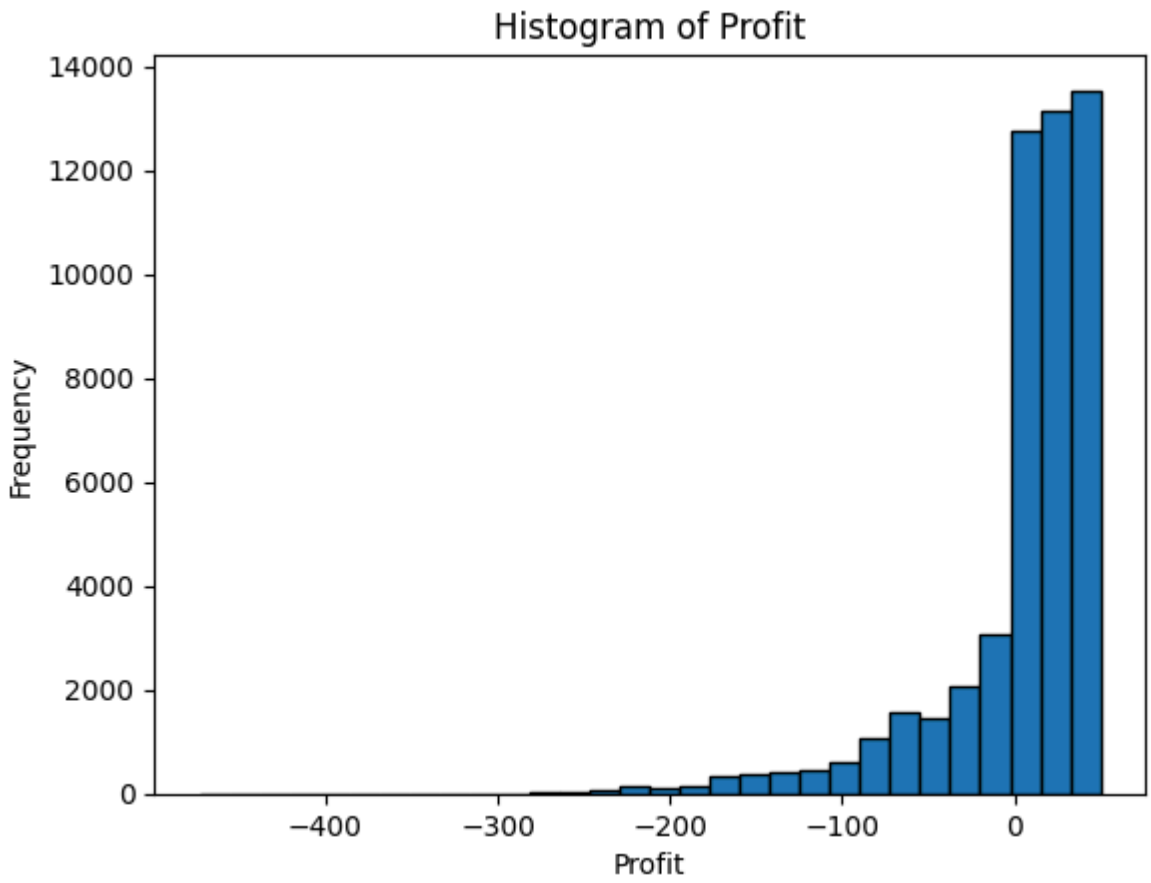
In [ ]:
```
import matplotlib.pyplot as plt
from statsmodels.graphics.gofplots import qqplot
```

In [ ]:
```
normality_check = new_data["Profit"]
qqplot(normality_check, line="s")
plt.show()
```



In [ ]:
```
#Normality check 2
# Plotting the histogram for Profit
plt.hist(new_data['Profit'], bins=30, edgecolor='black')
plt.title('Histogram of Profit')
plt.xlabel('Profit')
```

```
plt.ylabel('Frequency')
plt.show()
```



In [ ]:
```
# Perform Shapiro-Wilk test for normality
stat, p_value = stats.shapiro(new_data['Profit'])
print(f'Shapiro-Wilk Test: Stat={stat}, p-value={p_value}')

if p_value > 0.05:
    print("Data appears to be normally distributed (fail to reject H0).")
else:
    print("Data is not normally distributed (reject H0).")
```

```
Shapiro-Wilk Test: Stat=0.7532855720289295, p-value=1.0607240856753737e-121
Data is not normally distributed (reject H0).
```

```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: User
Warning: scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate.
Current N is 51290.
  res = hypotest_fun_out(*samples, **kwds)
```

In [ ]:
```
# Perform Levene's test for equal variances across regions
stat, p_value = stats.levene(*[new_data[new_data['Region'] == region]['Profit']
print(f'Levene Test: Stat={stat}, p-value={p_value}')

if p_value > 0.05:
    print("Variances are equal across groups (fail to reject H0).")
else:
    print("Variances are not equal across groups (reject H0).")
```

```
Levene Test: Stat=285.2959329767922, p-value=0.0
Variances are not equal across groups (reject H0).
```

In [ ]:
```
from scipy.stats import kruskal
stat, p_value = kruskal(
```

```
    profit_g1, profit_g2, profit_g3, profit_g4, profit_g5, profit_g6,
    profit_g7, profit_g8, profit_g9, profit_g10, profit_g11, profit_g12, profit_
)

print("\nKruskal-Wallis Test for Profit vs Random Region Group:")
print("Test statistic:", stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject H0: At least one random group has a different distribution of
else:
    print("Fail to reject H0: All random groups have the same distribution of pr
```

```
Kruskal-Wallis Test for Profit vs Random Region Group:
Test statistic: 13.052793806915727
P-value: 0.36521511190251527
Fail to reject H0: All random groups have the same distribution of profit.
```

In [ ]:
```
new_data = data.copy(deep=True)
```

In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import spearmanr

corr, p_value = spearmanr(new_data['Profit'], new_data['Region'])

print("Spearman's Correlation Coefficient:", corr)
print("P-value:", p_value)

if p_value < 0.05:
    print("The correlation is statistically significant.")
else:
    print("The correlation is not statistically significant.")
```

```
Spearman's Correlation Coefficient: 0.006462943638109985 5
P-value: 0.14328644195627804
The correlation is not statistically significant.
```

In [ ]:
```
data.head()
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	City
0	32298	CA-2012-124891	2012-07-31	2012-07-31	Same Day	RH-19495	Rick Hansen	Consumer	New York City
1	26341	IN-2013-77878	2013-02-05	2013-02-07	Second Class	JR-16210	Justin Ritter	Corporate	Wollongong
2	25330	IN-2013-71249	2013-10-17	2013-10-18	First Class	CR-12730	Craig Reiter	Consumer	Brisbane
3	13524	ES-2013-1579342	2013-01-28	2013-01-30	First Class	KM-16375	Katherine Murray	Home Office	Berlin
4	47221	SG-2013-4320	2013-11-05	2013-11-06	Same Day	RH-9495	Rick Hansen	Consumer	Dakar

5 rows × 29 columns

In [ ]:
```
data.describe(include = ['object'])
```

Out[ ]:

	Order ID	Ship Mode	Customer ID	Customer Name	Segment	City	State	Country
count	51290	51290	51290	51290	51290	51290	51290	51290
unique	25035	4	1590	795	3	3636	1094	147
top	CA-2014-100111	Standard Class	PO-18850	Muhammed Yedwab	Consumer	New York City	California	United States
freq	14	30775	97	108	26518	915	2001	9994

In [ ]:
```
data.drop('Random Region Group', axis=1, inplace=True, errors='ignore')
```

In [ ]:
```
numeric_data = data.select_dtypes(include=[float, int])
```

In [ ]:
```
kurtosis_values = numeric_data.kurt()
print(kurtosis_values)
```

```
Row ID              -1.200000
Sales              176.731200
Quantity             2.275889
Discount             0.716682
Profit               6.991184
Shipping Cost       50.020158
month               -0.977879
Profit Margin        6.991184
year                -1.222033
day                 -1.189482
Time Gap (Days)     -0.250011
dtype: float64
```

In [ ]:

```
#as previously it was clear that the dataset has numerous outliers in it.
#Outliers detection

num_new_data = data.select_dtypes(include = np.number)
num_new_data
```

Out[ ]:

	Row ID	Sales	Quantity	Discount	Profit	Shipping Cost	month	Profit Margin
0	32298	2309.650	7	0.0	33.000000	933.57	7	33.000000
1	26341	3709.395	9	0.1	-7.784693	923.63	2	-7.784693
2	25330	5175.171	9	0.1	17.776630	915.49	10	17.776630
3	13524	2892.510	5	0.1	-3.337586	910.16	1	-3.337586
4	47221	2832.960	8	0.0	10.996272	903.04	11	10.996272
...	...	...	...	...	...	...	...	...
51285	29002	65.100	5	0.0	6.912442	0.01	6	6.912442
51286	35398	0.444	1	0.8	-250.000000	0.01	6	-250.000000
51287	40470	22.920	3	0.0	49.000000	0.01	12	49.000000
51288	9596	13.440	2	0.0	17.857143	0.00	2	17.857143
51289	6147	61.380	3	0.0	2.932551	0.00	5	2.932551

51290 rows × 11 columns

In [ ]:

```
num_new_data
```

Out[ ]:

	Row ID	Sales	Quantity	Discount	Profit	Shipping Cost	month	Profit Margin
0	32298	2309.650	7	0.0	33.000000	933.57	7	33.000000
1	26341	3709.395	9	0.1	-7.784693	923.63	2	-7.784693
2	25330	5175.171	9	0.1	17.776630	915.49	10	17.776630
3	13524	2892.510	5	0.1	-3.337586	910.16	1	-3.337586
4	47221	2832.960	8	0.0	10.996272	903.04	11	10.996272
...	...	...	...	...	...	...	...	...
51285	29002	65.100	5	0.0	6.912442	0.01	6	6.912442
51286	35398	0.444	1	0.8	-250.000000	0.01	6	-250.000000
51287	40470	22.920	3	0.0	49.000000	0.01	12	49.000000
51288	9596	13.440	2	0.0	17.857143	0.00	2	17.857143
51289	6147	61.380	3	0.0	2.932551	0.00	5	2.932551

51290 rows × 11 columns

In [ ]:
```
num_new_data.drop('Row ID', axis = 1, inplace = True)
```

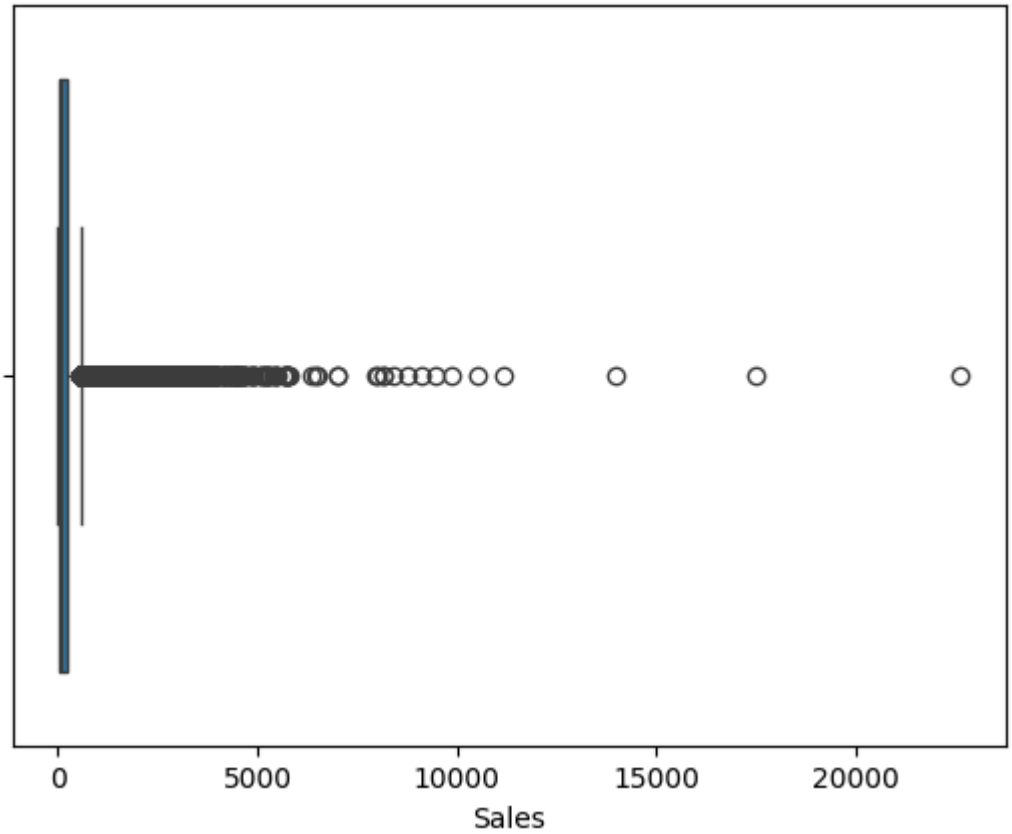
In [ ]:
```
num_new_data
```

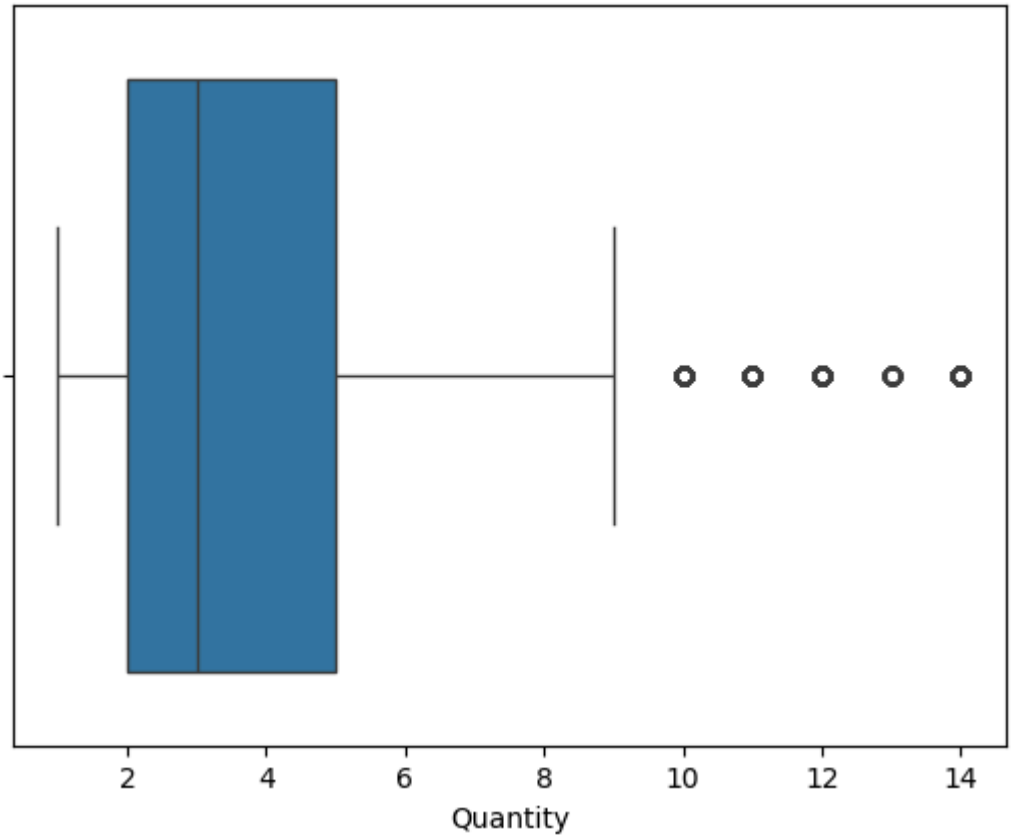
Out[ ]:

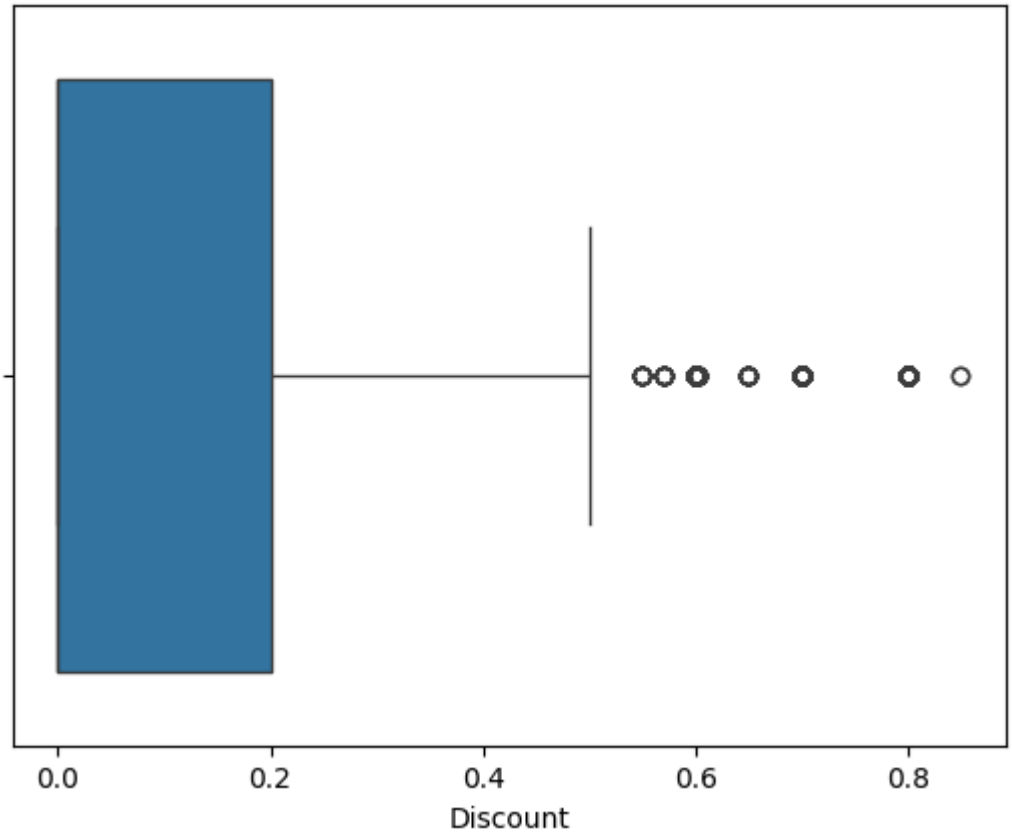
	Sales	Quantity	Discount	Profit	Shipping Cost	month	Profit Margin	year
0	2309.650	7	0.0	33.000000	933.57	7	33.000000	2012
1	3709.395	9	0.1	-7.784693	923.63	2	-7.784693	2013
2	5175.171	9	0.1	17.776630	915.49	10	17.776630	2013
3	2892.510	5	0.1	-3.337586	910.16	1	-3.337586	2013
4	2832.960	8	0.0	10.996272	903.04	11	10.996272	2013
...	...	...	...	...	...	...	...	...
51285	65.100	5	0.0	6.912442	0.01	6	6.912442	2014
51286	0.444	1	0.8	-250.000000	0.01	6	-250.000000	2014
51287	22.920	3	0.0	49.000000	0.01	12	49.000000	2013
51288	13.440	2	0.0	17.857143	0.00	2	17.857143	2012
51289	61.380	3	0.0	2.932551	0.00	5	2.932551	2012

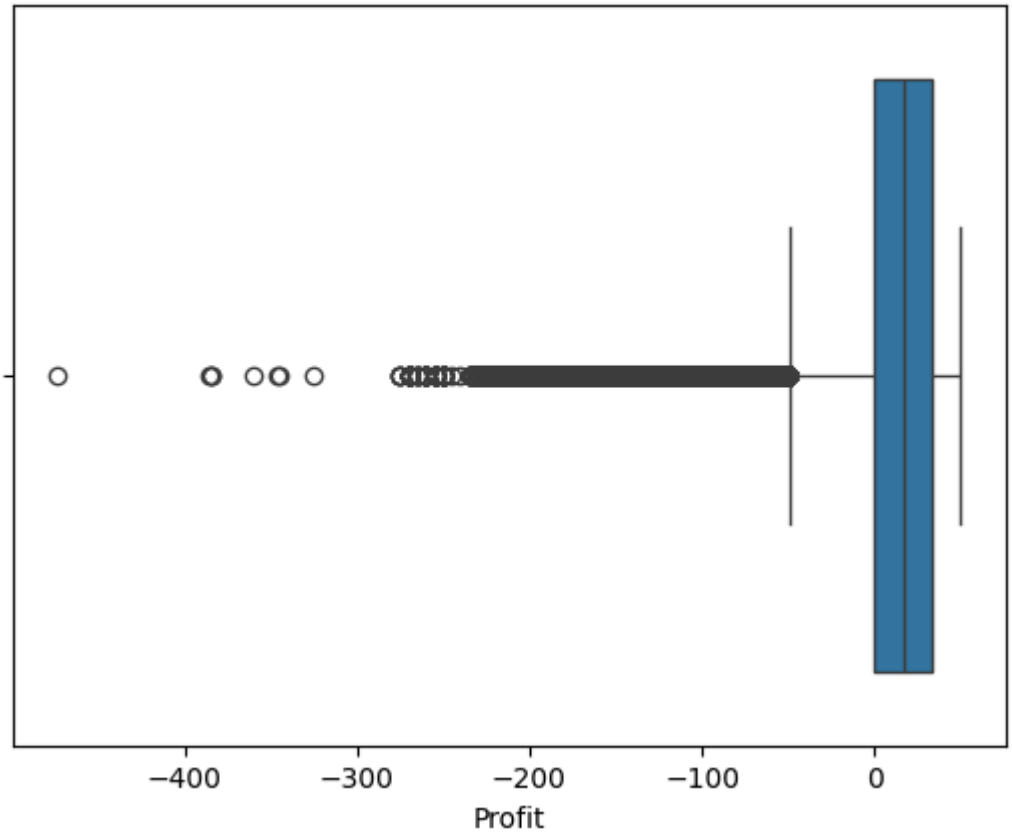
51290 rows × 10 columns

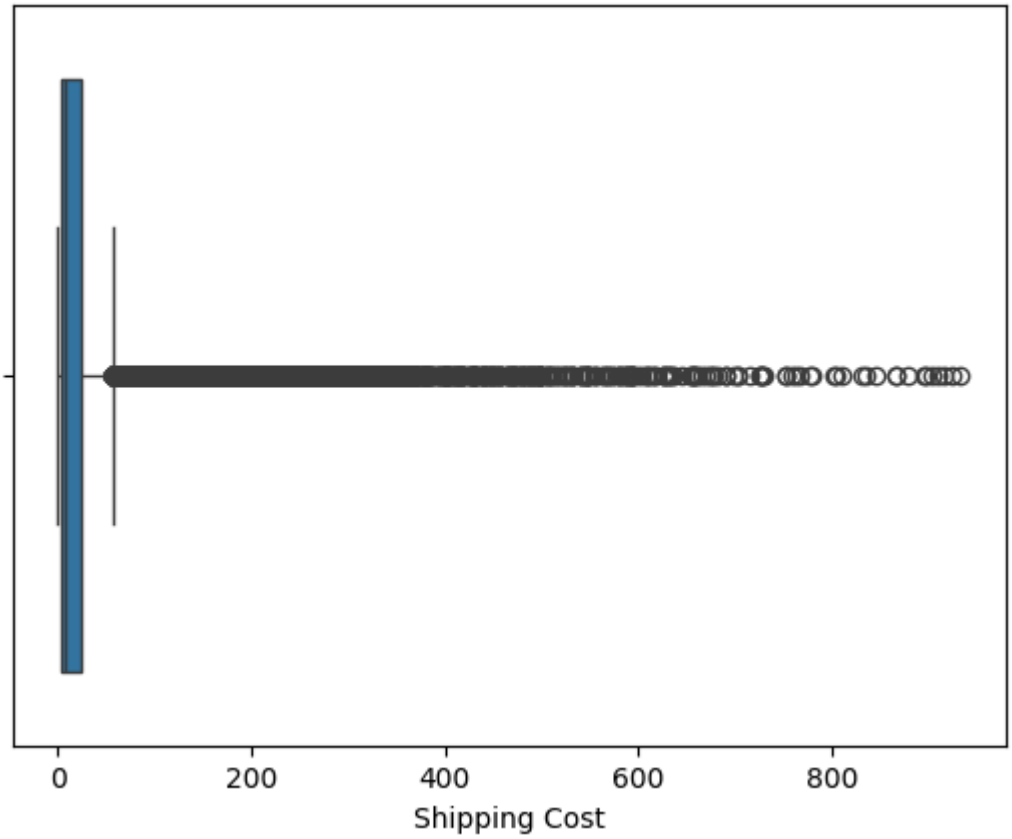
In [ ]:
```
for col in enumerate(num_new_data):
    sns.boxplot(x=col[1],data=num_new_data)
    plt.show()
```

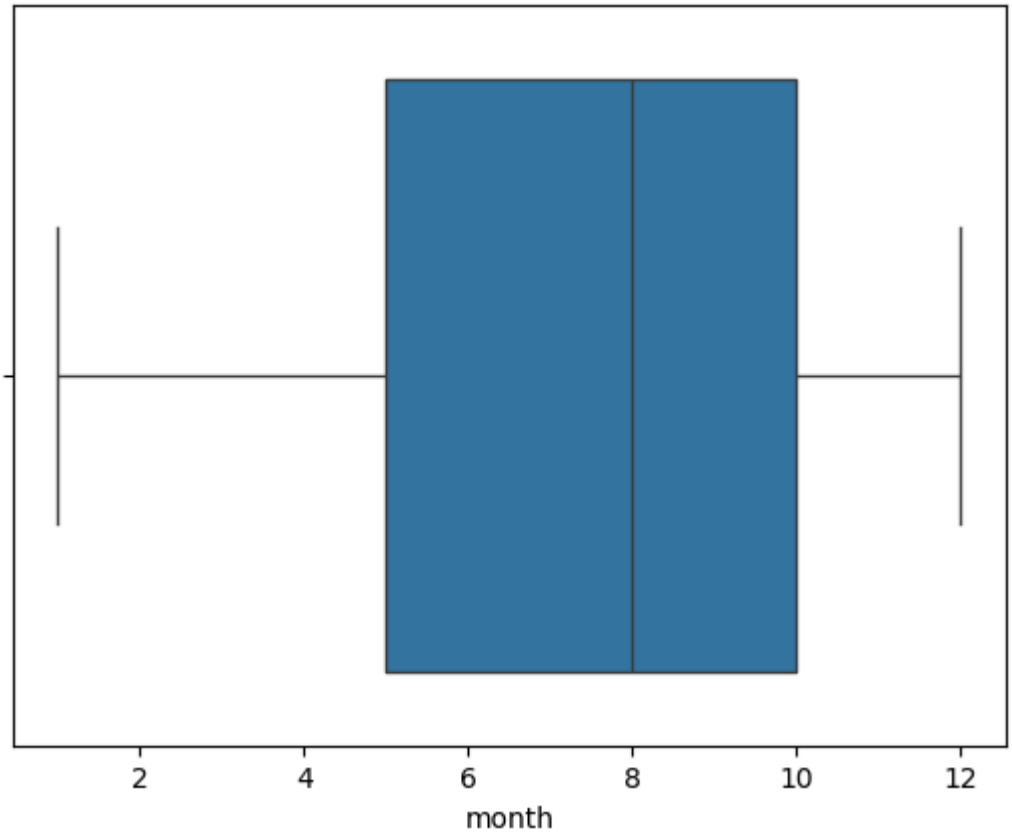


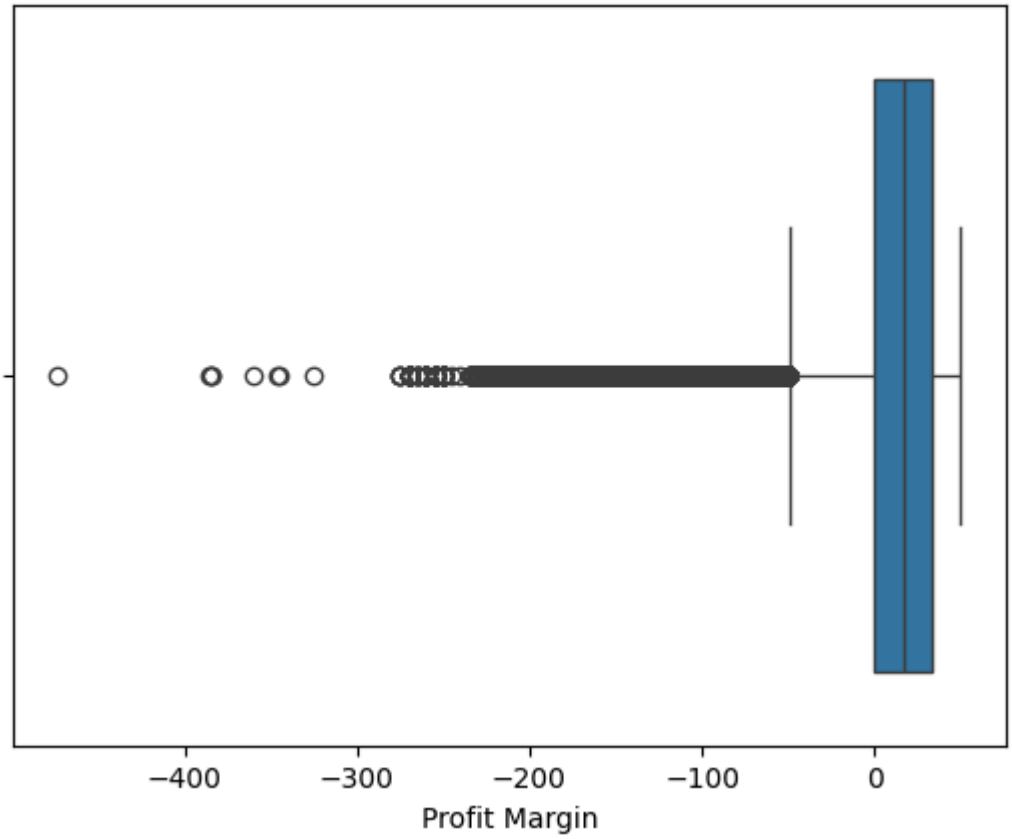


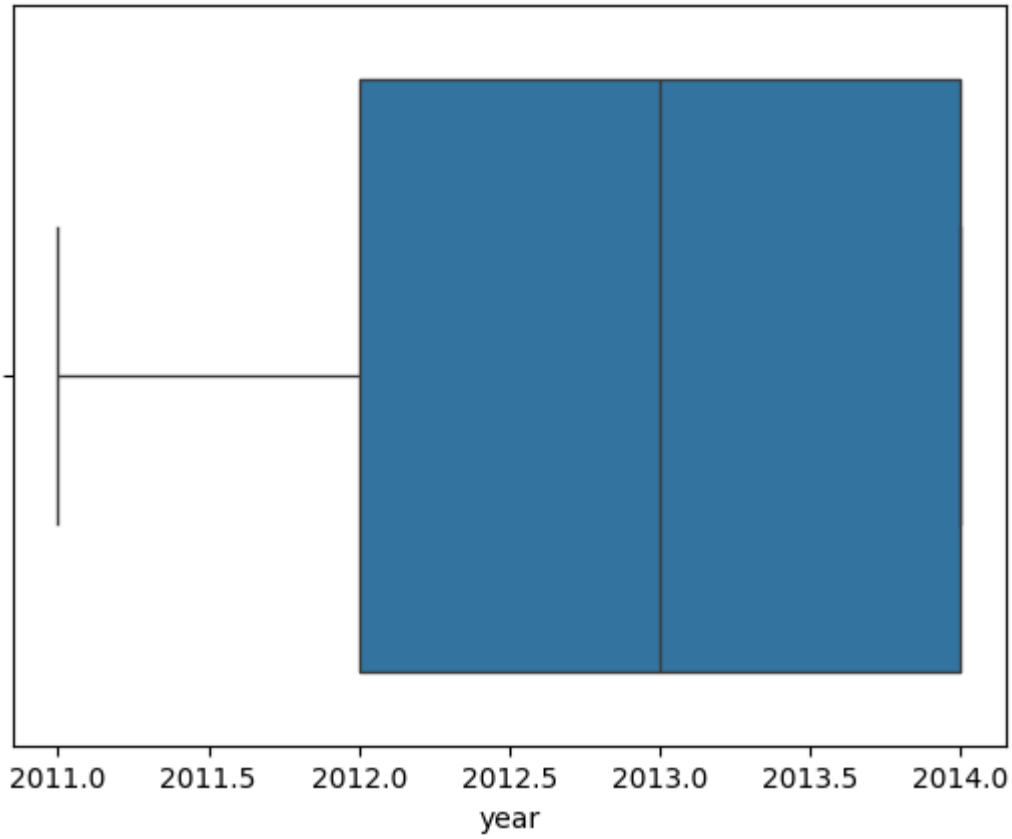

0.0
0.2
0.4
0.6
0.8
Discount

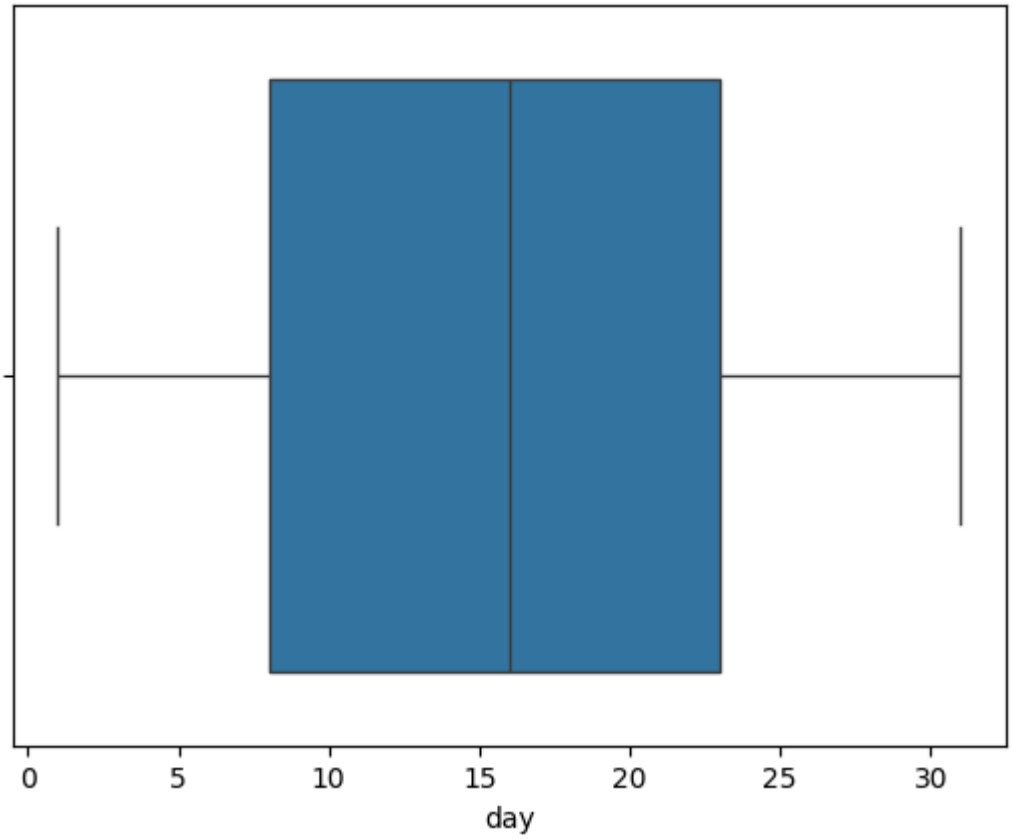

−400
−300
−200
−100
0
Profit

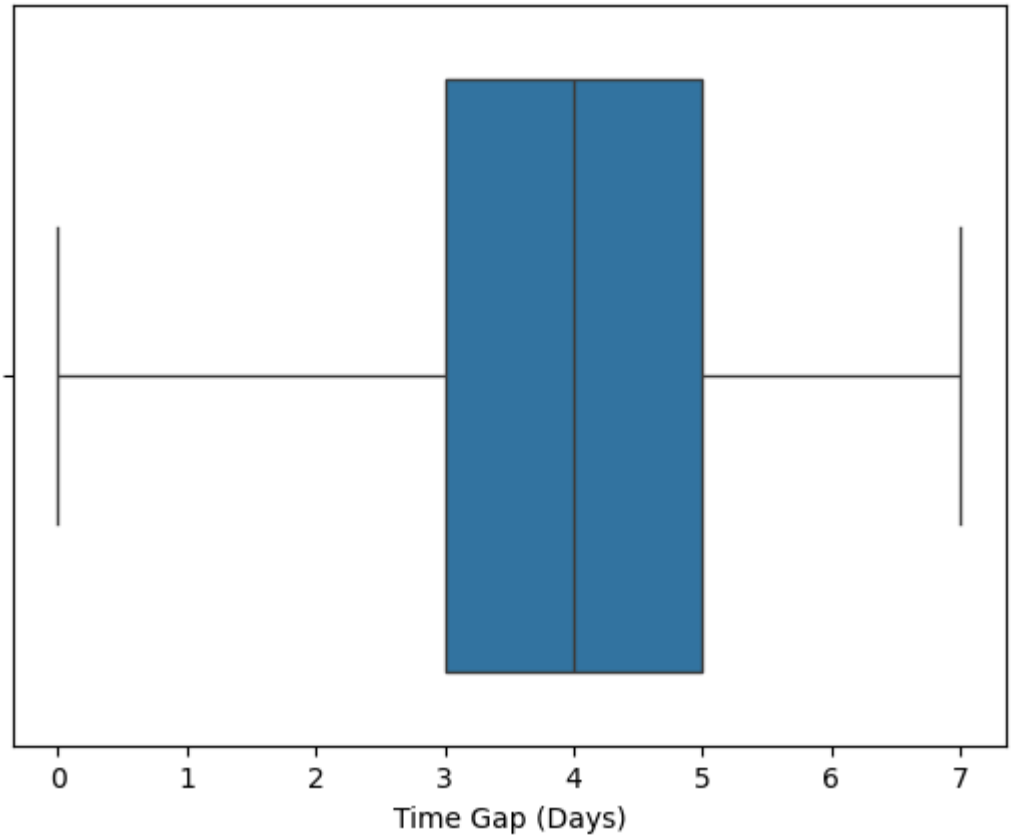

0
200
400
600
800
Shipping Cost


2
4
6
8
10
12
month


−400
−300
−200
−100
0
Profit Margin


2011.0
2011.5
2012.0
2012.5
2013.0
2013.5
2014.0
year





In [ ]:
```
#Using IQR
Q1 = num_new_data.quantile(0.25)

# obtain the third quartile
Q3 = num_new_data.quantile(0.75)

# obtain the IQR
IQR = Q3 - Q1
```

```
# print the IQR
print(IQR)
```

```
Sales              220.294575
Quantity             3.000000
Discount             0.200000
Profit              33.313913
Shipping Cost       21.840000
month                5.000000
Profit Margin       33.313913
year                 2.000000
day                 15.000000
Time Gap (Days)      2.000000
dtype: float64
```

In [ ]:
```
num_new_data.shape
```

Out[ ]:
```
(51290, 10)
```

In [ ]:
```
Q1 = num_new_data.quantile(0.25)
Q3 = num_new_data.quantile(0.75)
IQR = Q3 - Q1
```

In [ ]:
```
# Now perform the filtering
df_iqr = num_new_data[~((num_new_data < (Q1 - 1.5 * IQR)) | (num_new_data > (Q3
```

In [ ]: